

Publicly Auditable Garbled Circuit

San Ling¹, Chan Nam Ngo², Khai Hanh Tang¹, and Huaxiong Wang¹

¹ Nanyang Technological University
{lingsan,khaihanh.tang,hxwang}@ntu.edu.sg

² Ethereum Foundation
nam.ngo@ethereum.org

Abstract. Advanced cryptographic primitives such as zero-knowledge proofs (ZKPs) have seen an emerging trend in adoption by blockchain applications. However, ZKPs only allow a single prover to convince a verifier of a correct evaluation of a computation over the prover’s secret input. This is insufficient if the applications require shared private state from more than one party. In this case, one can adopt secure multiparty computation (MPC), which allows a set of mutually distrustful parties to jointly compute a global function while keeping their respective inputs private. Generic MPC, including secret-sharing and garbled-circuit techniques that allow the participants, without deep expertise in cryptography, to define the computation as a circuit, is an important tool in privacy-preserving protocols. Yet, generic MPC mostly focuses on malicious security in which the computed result is verifiable only among the computing parties, but not publicly verifiable. In blockchain applications, public verifiability is important, as the consensus protocol is not just among the computing parties but also public verifiers. Several works have attempted to bridge this gap, albeit not in the blockchain realm, i.e., publicly auditable MPC. Nevertheless, they are non-constant round protocols and only for secret-sharing MPC, i.e., round complexity scales linearly with the circuit multiplicative depth, while round latency is an important cost metric for blockchains. We address this problem by providing a Publicly Auditable Garbled Circuit protocol that is maliciously secure, publicly auditable, and constant-round. Our protocol is efficient, with minimal overhead in terms of the number of rounds, communication, and public transcript size.

Keywords: Garbled circuit · VOLE · public verifier · MPC

1 Introduction

Secure Multiparty Computation (MPC) allows n parties, namely, $P_1, \dots, P_n$, to securely evaluate a joint function $f(x_1, \dots, x_n)$ while keeping the secret input x_i private to its owner P_i . MPC comes with two main flavors of security [21], namely, *Semi-honest* and *Active Security*. The semi-honest model only guarantees security if the adversary follows the protocol, i.e., does not deviate from it but only tries to break privacy from the MPC transcript that is available to it. In contrast, the active security model is a stronger security setting that guarantees the security of the MPC protocol even if the adversary acts arbitrarily. Generic MPC is enabled through two approaches: Secret Sharing (SS) and Garbled Circuits [21] (GC). We focus on the latter due to its attractive property of constant-round complexity.

Verifiable MPC. We consider a somewhat “added-on” setting, in which an external party (called the Verifier) can verify the security of the MPC protocol, and we call protocols in this setting Verifiable MPC. Especially *Publicly Verifiable MPC* (formalized in [10]) where any external Verifier without any secret state can verify the protocol’s security (possibly with the aid from some Common Random String (CRS)). Its motivation is to run an MPC protocol among n parties, and the output will be acknowledged by the vast external verifiers of a blockchain consensus network.

Verifiability vs Auditability. To make an MPC verifiable, there are two approaches: *Verifiable Output* where the output of the MPC is made verifiable, while *Auditable Transcript* where the transcript of the MPC is made verifiable. We emphasize that the Verifiable Output flavor is independent, while the Auditable Transcript one depends on the MPC protocol. As an example, let us consider the *Shuffle* functionality. The *Shuffle* functionality runs between n parties $P_1, \dots, P_n$ each with a secret input $x_1, \dots, x_n$ starting as an ordered list of $O_0 = \{x_1, \dots, x_n\}$. The functionality runs in n rounds, in each round R_i , party P_i sends a permutation π_i to *Shuffle* in which the functionality will apply to obtain $O_i = \pi_i(O_{i-1})$. Finally, the functionality outputs O_n . *Verifiability* requires only that O_n is a permutation of O_0 . In contrast, *Auditability* asks for the correctness of the process, i.e., that O_n is a permutation of O_0 through a series of permutations $\pi_1, \dots, \pi_n$.

Changelog is presented in page 16.

Verifiable MPC. For Verifiable MPC, we can leverage Collaborative Zero-Knowledge Proof (co-ZKP, between $P_1, \dots, P_n$) to produce a ZK proof that the output $y = f(x_1, \dots, x_n)$ while keeping the $x_1, \dots, x_n$ private to $P_1, \dots, P_n$. In the literature, co-ZKP can be achieved by thresholdizing any existing ZKP, demonstrated by Collaborative Zero-Knowledge Succinct Argument of Knowledge (co-zkSNARKs) (c.f. [31]). There are potential Co-ZKPs based on other ZKPs such as: MPC-in-the-Head (MPCitH: seminal work IKOS07 [25], other MPCitH techniques [24,16,27,9,33,22,15,1,2]), VOLE-IZK (DVZK with fast prover time and small memory: [35,20,8,37,36,4,19,5]), or VOLE-in-the-Head (VOLEitH, [6]).³ Designated-Verifier MPC can leverage any private-coin Verifier (Co-)ZKP, Collaborative Verifiable MPC is achievable through thresholdizing the Verifier state in the private coin Verifier (Co-)ZKP, Multi-Verifier MPC can be obtained by running multiple Designated Verifier MPC instances, and Publicly Verifiable MPC can utilize any public coin verifier ZKP. Fortunately, all existing Co-ZKPs are constant-round protocols. Thus, the round complexity of the Verifiable MPC only depends on the underlying MPC protocol, i.e., a Secret-Sharing-based [21] (SS) Verifiable MPC will yield linear round complexity, while a Garbled-Circuit-based [21] (GC) Verifiable MPC will have constant-round complexity.

Auditable MPC. For Auditable MPC, we need the transcript to be auditable while keeping the privacy of the protocol execution. In general, for both SS-based and GC-based Auditable MPC, the transcript can be made auditable through Additively-Homomorphic Commitment (such as Pedersen, [7]) or Verifiable MAC [34] on each computation step. If the Commitment or MAC in the Auditable MPC can be made publicly verifiable, e.g., via some publicly verifiable Commitment or MAC [6], then we have Publicly Auditable MPC. Commitment-based approaches are expected to have worse concrete efficiency than MAC-based Auditable MPC due to the former operating on groups while the latter only operates on rings. One can consider Active Security a special case of Auditable MPC where the Verifier is a party in the MPC protocol. Hence, if the Active Security mechanism can be publicly verifiable, we will also obtain Publicly Auditable MPC. As in Verifiable MPC, an SS-based Auditable MPC (as in [7]) will yield linear round complexity, while a GC-based Auditable MPC can potentially yield a constant round complexity. This motivates our main research question:

Can we start from a GC-based actively secure protocol (such as WRK17 [27], KRRW18 [28], DILO22 [19], or CWYY23 [18]) and make such active security publicly verifiable and obtain constant-round publicly auditable MPC?

We answer this question affirmatively. We build on the basic blueprint of the actively secure Garbled Circuit protocol, namely, WRK17 [27], which relies on MAC. We then adopt VOLEitH [6] (with SoftSpokenOT [32]) to make the underlying MAC publicly auditable. We obtained a Constant-Round Publicly Auditable Garbled Circuit Protocol with inherited security and efficiency from both WRK17⁴ and VOLEitH. Notably, current shortest and efficient post-quantum signatures are constructed based on VOLEitH techniques [6,17,13,30]. In our protocol, considering practical parameters, our proof size is asymptotically about the transcript size of WRK protocol (see Section 5).

We implement and report the performance of our protocol at

<https://github.com/Crazy-Cryptographic-Buddies/pa-gc-rs>.

Paper organization. Section 2 contains the preliminaries. Section 3 presents a technical overview of our publicly auditable garbled circuit. Section 4 presents the functionality $\mathcal{F}_{\text{svole-2pc}}$, for generating VOLEitH correlations to enable publicly auditable garbled circuit, and its secure realization, namely, protocol $\Pi_{\text{svole-2pc}}$, as the intermediate components supporting protocol $\Pi_{\text{pa-2pc}}$, enabling publicly auditable garbled circuits. Based on the technical overview, Section 5 presents the construction of a publicly auditable garbled circuit by presenting the functionality $\mathcal{F}_{\text{pa-2pc}}$ and its secure realization, i.e., protocol $\Pi_{\text{pa-2pc}}$.

2 Preliminaries

Notations and conventions. We use $[a, b]$ to denote the set $\{a, a+1, \dots, b\}$ for any $a, b \in \mathbb{Z}$ satisfying $a \leq b$, and by $[b]$ to indicate $\{1, \dots, b\}$ for $b \in \mathbb{N}$ and $b \geq 1$. For $k \in [0, 3]$, let $(k_0, k_1) = \text{bin}_2(k)$ denote $(k_0, k_1) \in \{0, 1\}^2$ satisfying $k = k_0 + 2 \cdot k_1$. Throughout the paper, some notations contain both letters “A” and “B”; if $p \in \{A, B\}$, we denote by $\bar{p}$ the other element in $\{A, B\}$.

³ VOLEitH is the public coin Verifier version of VOLE-IZK

⁴ We disregard the improvements in KRRW18 [28], DILO22 [19], or CWYY23 [18], due to non-trivial composition of such improvements and VOLEitH; for example they may require Learning-Parity-With-Noise assumption while SoftSpokenOT is incompatible with LPN. We leave such composition to future work.

2.1 Authenticated Bits via VOLE Correlations

Let $\mathcal{P}_A$ and $\mathcal{P}_B$ be two communicating parties. For $p \in \{A, B\}$, assume that $\mathcal{P}_p$ holds a secret bit $b \in \mathbb{F}_2$ while $\mathcal{P}_{\bar{p}}$ holds a global value $\Delta_{\bar{p}}$. Then, a VOLE correlation for authenticating the bit b w.r.t. $\Delta_{\bar{p}}$ is a correlation where $\mathcal{P}_p$ additionally holds $M[b]$ while $\mathcal{P}_{\bar{p}}$ additionally holds $K[b]$ satisfying $M[b] = K[b] \oplus b \cdot \Delta_{\bar{p}}$.

2.2 VOLE-in-the-Head (VOLEitH)

VOLEitH from SoftSpokenOT [32] allows two parties to generate an authenticated bit, such that each party only interacts locally with the random oracle (RO). This leads to publicly verifiable ZKPs [6,17,13] which recently attracted a lot of attention from the research community.

Functionality $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}, \mathcal{V}, \tau, N}$. We recall the subspace VOLE functionality $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}, \mathcal{V}, \tau, N}$ [6], for VOLEitH, adapted from [17], in Figure 3.

VOLEitH from SoftSpokenOT. According to [32], an all-but-one OT correlation is equivalent to a subfield VOLE correlation over $\mathbb{F}_{2^\tau}$ for some $\tau \in \mathbb{N}$. Specifically, let $t_x \in \{0, 1\}$ for all $x \in \mathbb{F}_{2^\tau}$. Based on such an equivalence, SoftSpokenOT works by the observation that

$$\underbrace{\bigoplus_{x \in \mathbb{F}_{2^\tau} \setminus \{\nabla\}} t_x \cdot (\nabla \oplus x)}_{K'[b]} = \underbrace{\bigoplus_{x \in \mathbb{F}_{2^\tau}} t_x \cdot x}_{M'[b]} \oplus \underbrace{\bigoplus_{x \in \mathbb{F}_{2^\tau}} t_x}_{b} \cdot \nabla. \quad (1)$$

We can set $b = \bigoplus_{x \in \mathbb{F}_{2^\tau}} t_x$, $M'[b] = \bigoplus_{x \in \mathbb{F}_{2^\tau}} t_x \cdot x$ and $K'[b] = \bigoplus_{x \in \mathbb{F}_{2^\tau} \setminus \{\nabla\}} t_x \cdot (\nabla \oplus x)$. It follows that $M'[b] = K'[b] \oplus b \cdot \nabla$. Hence, we can run VOLEitH (from SoftSpokenOT) to generate an authenticated random string $\mathbf{u} = (u_i)_{i=1}^N \in \{0, 1\}^N$ for some $N \in \mathbb{N}$ such that it generates $\mathbf{v} = (M'[u_i])_{i=1}^N \in \mathbb{F}_{2^\tau}^N$ and $\mathbf{w} = (K'[u_i])_{i=1}^N \in \mathbb{F}_{2^\tau}^N$ satisfying $\mathbf{v} = \mathbf{w} \oplus \mathbf{u} \cdot \nabla$ as expected in functionality $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}, \mathcal{V}, \tau, N}$. This is placed in Figure 1 as protocol $\Pi_{\text{sVOLE}}^{\mathcal{P}, \mathcal{V}, \tau, N}$ that securely realizes $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}, \mathcal{V}, \tau, N}$.

Remark 1. In Figure 1, we can separate into two phases as follows. The first phase (at step 1) is before knowing ∇ and the keys. The second phase (after step 1) is after knowing ∇ and the keys.

Hence, during the paper, we may indicate whether ∇ and the keys are known or unknown to indicate whether the respective running protocol is in which phase between the two phases mentioned above.

This functionality runs between $\mathcal{P}$ and $\mathcal{V}$ with $\tau, N \in \mathbb{N}$:

1. $\mathcal{P}$ employs a pseudo-random number PRG to generate $\mathbf{t}_x \in \{0, 1\}^N \forall x \in \mathbb{F}_{2^\tau}$. $\mathcal{P}$ can compute $\mathbf{u} = \bigoplus_{x \in \mathbb{F}_{2^\tau}} \mathbf{t}_x$, $\mathbf{v} = (M'[u_i])_{i=1}^N = \bigoplus_{x \in \mathbb{F}_{2^\tau}} \mathbf{t}_x \cdot x$. Then, $\mathcal{P}$ uses a vector commitment scheme (realizing $\mathcal{F}_{\text{VC}}^{\tau, N}$ in Figure 2) to commit to all $\mathbf{t}_x \forall x \in \mathbb{F}_{2^\tau}$ and sends the commitment to $\mathcal{V}$.
2. $\mathcal{V}$ samples and sends $\nabla \xleftarrow{\$} \mathbb{F}_{2^\tau}$ to $\mathcal{P}$.
3. $\mathcal{P}$ then opens to $\mathcal{V}$ all $\mathbf{t}_x$ for $x \in \mathbb{F}_{2^\tau} \setminus \{\nabla\}$.
4. $\mathcal{P}, \mathcal{V}$ computes $\mathbf{w} = (K'[u_i])_{i \in [N]} = \bigoplus_{x \in \mathbb{F}_{2^\tau} \setminus \{\nabla\}} \mathbf{t}_x \cdot (\nabla \oplus x)$.
5. $\mathcal{V}$ accepts if the openings by $\mathcal{P}$ are correct and $(M'[u_i])_{i \in [N]} = (K'[u_i])_{i \in [N]} \oplus \mathbf{u} \cdot \nabla$.

Fig. 1. Protocol $\Pi_{\text{sVOLE}}^{\mathcal{P}, \mathcal{V}, \tau, N}$ for VOLEitH.

Amplifying soundness. Following [17], to amplify soundness, one may need to repeat the protocol κ times independently for some $\kappa \in \mathbb{N}$. Specifically, a bit b can be authenticated using $(M'_j[b])_{j=1}^\kappa$ and $(K'_j[b])_{j=1}^\kappa$ such that $M'_j[b] = K'_j[b] \oplus b \cdot \nabla_j \forall j \in [\kappa]$.

Sub-protocol $\Pi_{\text{fix}}^{\mathcal{P}, \mathcal{V}}$ for generating VOLEitH correlations for a fixed bit. $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}, \mathcal{V}, \tau, N}$ is only for generating VOLEitH correlations for random bits. To generate the κ VOLEitH correlations for a fixed bit b , one can apply $\Pi_{\text{fix}}^{\mathcal{P}, \mathcal{V}}$ [8] as follows. $\mathcal{P}$ generates VOLEitH tags $(M'_j[\bar{b}_j])_{j=1}^\kappa$ and random bits $(\bar{b}_j)_{j=1}^\kappa$ from step 1 of $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}, \mathcal{V}, \tau, N}$, sends $(d_j)_{j=1}^\kappa$, where $d_j = \bar{b}_j \oplus b$, to $\mathcal{V}$, and sets $M'_j[b] := M'_j[\bar{b}_j] \forall j \in [\kappa]$. When achieving $(\nabla_j)_{j=1}^\kappa$ and $(K'_j[\bar{b}_j])_{j=1}^\kappa$ from

This functionality runs between $\mathcal{P}$ and $\mathcal{V}$ with $\tau, N \in \mathbb{N}$:

1. Upon receiving (commit) from $\mathcal{P}$ and $\mathcal{V}$, sample $\mathbf{t}_x \xleftarrow{\$} \{0, 1\}^N \forall x \in \mathbb{F}_{2^\tau}$. If $\mathcal{P}$ is corrupted, receive $\mathbf{t}_x \forall x \in \mathbb{F}_{2^\tau}$ from $\mathcal{P}$. Then, send (done) to both $\mathcal{P}$ and $\mathcal{V}$.
2. Upon receiving (get, ∇) from $\mathcal{V}$, send $(\mathbf{t}_x)_{x \in \mathbb{F}_{2^\tau} \setminus \{\nabla\}}$ to both $\mathcal{P}$, $\mathcal{V}$.

Fig. 2. Functionality $\mathcal{F}_{\text{VC}}^{\tau, N}$.

This functionality is parameterized by the field $\mathbb{F}_2$ and its extension $\mathbb{F}_{2^\tau}$ for some $\tau \in \mathbb{N}$. N denotes the number of random sVOLE correlations. It works as follows.

1. Upon receiving (init) from $\mathcal{P}$ and $\mathcal{V}$, do
 - (a) $\nabla \xleftarrow{\$} \mathbb{F}_{2^\tau}$, $\mathbf{u} \xleftarrow{\$} \mathbb{F}_2^N$, $\mathbf{v} \xleftarrow{\$} \mathbb{F}_{2^\tau}^N$, $\mathbf{w} := \mathbf{v} \oplus \mathbf{u} \cdot \nabla$.
 - (b) If $\mathcal{P}$ is corrupted, receive $\mathbf{u}, \mathbf{v}$ from $\mathcal{P}$ and compute $\mathbf{w} := \mathbf{v} \oplus \mathbf{u} \cdot \nabla$.
 - (c) If $\mathcal{V}$ is corrupted, receive $\nabla, \mathbf{w}$ from $\mathcal{V}$ and compute $\mathbf{v} := \mathbf{w} \oplus \mathbf{u} \cdot \nabla$.
 - (d) Send $(\mathbf{u}, \mathbf{v})$ to $\mathcal{P}$.
2. Upon receiving (get) from $\mathcal{P}$ and $\mathcal{V}$, send $(\nabla, \mathbf{w})$ to both parties.

Note: We use Δ for VOLE correlations (derived from WRK [34]) while ∇ is used for public verifiability via VOLEitH. We call those w.r.t. ∇ the *VOLEitH correlations*. Moreover, for VOLEitH, we call $M'[b]$ and $K'[b]$, for a bit b , the VOLEitH tag and key of b , respectively. Moreover, ∇ is VOLEitH global key.

Fig. 3. The subspace VOLE functionality $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}, \mathcal{V}, \tau, N}$.

step 2 of $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}, \mathcal{V}, \tau, N}$, $\mathcal{V}$ can determine $K'_j[b] := K'_j[\bar{b}_j] \oplus d_j \cdot \nabla_j$ for $j \in [\kappa]$. Via $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}, \mathcal{V}, \tau, N}$, since $K'_j[\bar{b}_j] = M'_j[\bar{b}_j] \oplus \bar{b}_j \cdot \nabla_j$ for $j \in [\kappa]$, one can easily verify that $K'_j[b] = M'_j[b] \oplus b \cdot \nabla_j$.

Instantiations of VOLEitH. As in [6], to instantiate VOLEitH, we can realize $\mathcal{F}_{\text{VC}}^{\tau, N}$ by a GGM tree (previously used in [27, 11, 14]). Here, we skip the details of such realization and refer readers to [6]. Regarding the cost of vector commitments and their all-but-one openings, we denote by $\text{costvc}_{\text{com}}^{\tau, N}$ to indicate the size of the vector commitment to a vector of length 2^τ bits and by $\text{costvc}_{\text{open}}^{\tau, N}$ to indicate the size of its all-but-one opening. We also denote by $\text{costvc}_{\text{total}}^{\tau, N}$ to indicate the total cost $\text{costvc}_{\text{com}}^{\tau, N} + \text{costvc}_{\text{open}}^{\tau, N}$. Hence, when repeating κ times, the total cost is $\kappa \cdot (\text{costvc}_{\text{com}}^{\tau, N} + \text{costvc}_{\text{open}}^{\tau, N} + \tau) = \kappa \cdot (\text{costvc}_{\text{total}}^{\tau, N} + \tau)$ bits of communication where the extra $\kappa \cdot \tau$ bits is for global challenges.

Removing Interactions. As VOLEitH is public-coin, i.e., $\mathcal{V}$ has no secrets and ∇ is sampled and made public by $\mathcal{V}$. Hence, VOLEitH can be made non-interactive by using Fiat-Shamir transformation [23] to generate ∇ from the RO. When repeating κ times, the RO generates κ global values (say, e.g., $(\nabla_j)_{j \in [\kappa]}$) instead.

2.3 Beaver's Multiplication Trick

Recall Beaver's multiplication trick [12], adapted for VOLEitH, which is used to prove that $x \cdot y = z$ where $x, y, z \in \{0, 1\}$ are authenticated via VOLEitH correlations (c.f. Section 2), i.e., $M'_j[x] = K'_j[x] \oplus x \cdot \nabla_j$, $M'_j[y] = K'_j[y] \oplus y \cdot \nabla_j$, and $M'_j[z] = K'_j[z] \oplus z \cdot \nabla_j$ for $j \in [\kappa]$. Then, to prove $x \cdot y = z$, this trick requires that the prover already has $(a_j)_{j=1}^\kappa, (b_j)_{j=1}^\kappa, (c_j)_{j=1}^\kappa \in \{0, 1\}^\kappa$ authenticated via VOLEitH correlations, i.e.,

$$M'_j[a_j] = K'_j[a_j] \oplus a_j \cdot \nabla_j, M'_j[b_j] = K'_j[b_j] \oplus b_j \cdot \nabla_j, \text{ and } M'_j[c_j] = K'_j[c_j] \oplus c_j \cdot \nabla_j,$$

s.t. $a_j \cdot b_j = c_j$ for $j \in [\kappa]$.

- When ∇_j and the VOLEitH keys are unknown, $\mathcal{P}$ first determines $d_j := x \oplus a_j$, $M'_j[d_j] := M'_j[x] \oplus M'_j[a_j]$, $e_j := y \oplus b_j$, $M'_j[e_j] := M'_j[y] \oplus M'_j[b_j]$, $\tilde{z}_j := z \oplus c_j \oplus d_j \cdot b_j \oplus e_j \cdot a_j$, and $M'_j[\tilde{z}_j] := M'_j[z] \oplus M'_j[c_j] \oplus d_j \cdot M'_j[b_j] \oplus e_j \cdot M'_j[a_j]$. $\mathcal{P}$ sends $(d_j)_{j=1}^\kappa, (e_j)_{j=1}^\kappa, (\tilde{z}_j)_{j=1}^\kappa, (M'_j[d_j])_{j=1}^\kappa, (M'_j[e_j])_{j=1}^\kappa, (M'_j[\tilde{z}_j])_{j=1}^\kappa$ to $\mathcal{V}$.
- When ∇_j and the VOLEitH keys $K'_j[x], K'_j[y], K'_j[z], K'_j[a_j], K'_j[b_j], K'_j[c_j]$, for $j \in [\kappa]$, are known, the verifier can determine $K'_j[d_j] := K'_j[x] \oplus K'_j[a_j]$, $K'_j[e_j] := K'_j[y] \oplus K'_j[b_j]$, and $K'_j[\tilde{z}_j] := K'_j[z] \oplus K'_j[c_j] \oplus d_j \cdot K'_j[b_j] \oplus e_j \cdot K'_j[a_j]$. Then, verifier checks whether $M'_j[d_j] = K'_j[d_j] \oplus d_j \cdot \nabla_j$, $M'_j[e_j] = K'_j[e_j] \oplus e_j \cdot \nabla_j$, $M'_j[\tilde{z}_j] = K'_j[\tilde{z}_j] \oplus \tilde{z}_j \cdot \nabla_j$, and $\tilde{z}_j \oplus d_j \cdot e_j = 0$ for $j \in [\kappa]$.

Here, checking $M'_j[d_j] = K'_j[d_j] \oplus d_j \cdot \nabla_j$, $M'_j[e_j] = K'_j[e_j] \oplus e_j \cdot \nabla_j$, $M'_j[\tilde{z}_j] = K'_j[\tilde{z}_j] \oplus \tilde{z}_j \cdot \nabla_j$ aims to guarantee that d_j , e_j , and $\tilde{z}_j$ are correctly computed from the witnesses of $\mathcal{P}$. The remaining check, i.e., $\tilde{z}_j \oplus d_j \cdot e_j = 0$, is due to

$$\tilde{z}_j \oplus d_j \cdot e_j = (z \oplus c_j \oplus (x \oplus a_j) \cdot b_j \oplus (y \oplus b_j) \cdot a_j) \oplus (x \oplus a_j) \cdot (y \oplus b_j) = z \oplus x \cdot y$$

since $a_j \cdot b_j = c_j$. It can be seen that, if $z = x \cdot y$ then $z \oplus c_j \oplus d_j \cdot b_j \oplus e_j \cdot a_j = c_j \cdot d_j$.

2.4 Commitment Schemes

We recall the definition of commitment schemes (adapted from [26]) in the following Definition 1.

Definition 1. A commitment scheme $\mathcal{COM}$ is a triple of algorithms $\mathcal{COM} = (\text{KeyGen}, \text{Com}, \text{Verify})$ defined as follows:

KeyGen(1^λ) $\rightarrow$ **ck**: On input the security parameter 1^λ , this randomized algorithm returns a commitment key **ck**.

Com(**ck**, M) $\rightarrow$ (**cm**, **rand**): On inputs a commitment key and a message M , this algorithm returns a commitment **cm** to M and a randomness **rand**.

Verify(**ck**, M , **cm**, **rand**) $\rightarrow \{0, 1\}$: This is the verification algorithm to verify whether **cm** is a valid commitment to M w.r.t. **rand**. If so, this algorithm returns 1; otherwise, it returns 0. In case that **cm** opens to M .

$\mathcal{COM}$ is secure if it satisfies the following properties:

Correctness. For any **ck** output by **KeyGen**, any (**cm**, **rm**) output by **Com**(**ck**, M), it always holds that

$$\text{Verify}(\text{ck}, M, \text{cm}, \text{rand}) = 1.$$

Computational Binding. For any PPT adversary $\mathcal{A}$, any **ck** output by **KeyGen**, $\mathcal{A}$ has negligible probability to succeed in finding (**cm**, M , **rand**, M' , **rand'**) satisfying $\text{Verify}(\text{ck}, M, \text{cm}, \text{rand}) = 1$, $\text{Verify}(\text{ck}, M', \text{cm}, \text{rand}') = 1$, and $M \neq M'$.

Statistical Hiding. For any adversary $\mathcal{A}$, the following two distributions are statistically indistinguishable.

$$\left\{ \text{cm} \left| \begin{array}{l} \text{ck} \leftarrow \text{KeyGen}(1^\lambda), \\ (M_0, M_1, \text{aux}) \leftarrow \mathcal{A}(\text{ck}), \\ (\text{cm}, \text{rand}) \leftarrow \text{Com}(\text{ck}, M_0) \end{array} \right. \right\} \text{ and } \left\{ \text{cm} \left| \begin{array}{l} \text{ck} \leftarrow \text{KeyGen}(1^\lambda), \\ (M_0, M_1, \text{aux}) \leftarrow \mathcal{A}(\text{ck}), \\ (\text{cm}, \text{rand}) \leftarrow \text{Com}(\text{ck}, M_1) \end{array} \right. \right\}.$$

3 Technical Overview

We first recall the components from authenticated garbling in the WRK protocol [34] in Section 3.1. Then, Section 3.2 discusses our technique for making it publicly auditable.

Remark 2. For a Boolean circuit $\mathcal{C}$, $\mathcal{I}_A$ and $\mathcal{I}_B$ denote the set of input wires held by $\mathcal{P}_A$ and $\mathcal{P}_B$, respectively, $\mathcal{I}_W$ denotes the set containing output wires of all AND gates, and $\mathcal{I}_O$ denotes the set of output wires of the circuit. Hence, we write $\text{spec}(\mathcal{C}) = (\mathcal{I}_A, \mathcal{I}_B, \mathcal{I}_W, \mathcal{I}_O)$. Denote $W = |\mathcal{I}_W|$.

3.1 Components from Authenticated Garbling by WRK Protocol

The WRK protocol [34] is run by $\mathcal{P}_A$ (garbler) and $\mathcal{P}_B$ (evaluator). The detailed description of WRK protocol is deferred to Appendix A. Consider a circuit containing AND ($\wedge$) and XOR ($\oplus$) gates. We denote each gate by the tuple $(\alpha, \beta, \gamma, \text{op})$ where $\text{op} \in \{\wedge, \oplus\}$ is the operation, and α , β , and γ are the indices of the left, right, and output wires, respectively. $z_w \in \{0, 1\}$ denotes the value at wire w and is masked by $\lambda_w \in \{0, 1\}$ to become $\hat{z}_w = z_w \oplus \lambda_w$. Additionally, λ_w is split into shared random masks $\lambda_{A,w}$ and $\lambda_{B,w}$ held by $\mathcal{P}_A$ and $\mathcal{P}_B$, respectively, s.t. $\lambda_w = \lambda_{A,w} \oplus \lambda_{B,w}$. The input wires are split into two disjoint sets $\mathcal{I}_A$ and $\mathcal{I}_B$ s.t. $\mathcal{P}_A$ (resp., $\mathcal{P}_B$) knows z_w for $w \in \mathcal{I}_A$ (resp., $w \in \mathcal{I}_B$). Therefore, in the beginning (at the phase **Input Preprocessing**), $\mathcal{P}_B$ (resp., $\mathcal{P}_A$) must send $\lambda_{B,w}$ (resp., $\lambda_{A,w}$) to $\mathcal{P}_A$ (resp., $\mathcal{P}_B$) for computing $\hat{z}_w$ for $w \in \mathcal{I}_A$ (resp., $w \in \mathcal{I}_B$). $\mathcal{P}_A$ and $\mathcal{P}_B$ then broadcast all $\hat{z}_w$ for $w \in \mathcal{I}_A \cup \mathcal{I}_B$. Evaluation follows a topological order:

XOR gate $(\alpha, \beta, \gamma, \oplus)$. Having $\hat{z}_\alpha$ and $\hat{z}_\beta$, each party can locally compute “for free” (i.e., without interaction) $\hat{z}_\gamma := \hat{z}_\alpha \oplus \hat{z}_\beta = \underbrace{(z_\alpha \oplus z_\beta)}_{z_\gamma} \oplus \underbrace{(\lambda_\alpha \oplus \lambda_\beta)}_{\lambda_\gamma}$.

AND gate $(\alpha, \beta, \gamma, \wedge)$. Having $\hat{z}_\alpha$ and $\hat{z}_\beta$, observe that

$$\begin{aligned}\hat{z}_\alpha \cdot \hat{z}_\beta &= (z_\alpha \oplus \lambda_\alpha) \cdot (z_\beta \oplus \lambda_\beta) \\ &= z_\alpha \cdot z_\beta \oplus \lambda_\alpha \cdot z_\beta \oplus \lambda_\beta \cdot z_\alpha \oplus \lambda_\alpha \cdot \lambda_\beta \\ &= z_\alpha \cdot z_\beta \oplus \lambda_\alpha \cdot \underbrace{(z_\beta \oplus \lambda_\beta)}_{\hat{z}_\beta} \oplus \lambda_\beta \cdot \underbrace{(z_\alpha \oplus \lambda_\alpha)}_{\hat{z}_\alpha} \oplus \lambda_\alpha \cdot \lambda_\beta.\end{aligned}$$

Hence, $z_\alpha \cdot z_\beta \oplus \lambda_\gamma = \lambda_\alpha \cdot \lambda_\beta \oplus \lambda_\gamma \oplus \lambda_\alpha \cdot \hat{z}_\beta \oplus \lambda_\beta \cdot \hat{z}_\alpha \oplus \hat{z}_\alpha \cdot \hat{z}_\beta$. Notice that $\lambda_\alpha \cdot \hat{z}_\beta = \lambda_{A,\alpha} \cdot \hat{z}_\beta \oplus \lambda_{B,\alpha} \cdot \hat{z}_\beta$, $\lambda_\beta \cdot \hat{z}_\alpha = \lambda_{A,\beta} \cdot \hat{z}_\alpha \oplus \lambda_{B,\beta} \cdot \hat{z}_\alpha$, and $\lambda_\gamma = \lambda_{A,\gamma} \oplus \lambda_{B,\gamma}$ s.t. $\mathcal{P}_A$ can compute $\lambda_{A,\alpha} \cdot \hat{z}_\beta$ and $\lambda_{A,\beta} \cdot \hat{z}_\alpha$ while $\mathcal{P}_B$ can compute $\lambda_{B,\alpha} \cdot \hat{z}_\beta$ and $\lambda_{B,\beta} \cdot \hat{z}_\alpha$.

Regarding $\lambda_\alpha \cdot \lambda_\beta$, WRK [34] introduces additional shared random masks $\lambda'_{A,\gamma}$ and $\lambda'_{B,\gamma}$ held by $\mathcal{P}_A$ and $\mathcal{P}_B$, respectively. These masks satisfy $\lambda'_{A,\gamma} \oplus \lambda'_{B,\gamma} = \lambda_\alpha \cdot \lambda_\beta$. Then, $\mathcal{P}_A$ can construct the 4-row garbled table encrypting the four possibilities of $(\hat{z}_\alpha, \hat{z}_\beta)$ where the k -th row is

$$\lambda_{A,\gamma,k} = \lambda'_{A,\gamma} \oplus \lambda_{A,\gamma} \oplus k_0 \cdot \lambda_{A,\beta} \oplus k_1 \cdot \lambda_{A,\alpha} \quad (2)$$

for $k \in [0, 3]$ with $(k_0, k_1) = \text{bin}_2(k)$. Regarding $\mathcal{P}_B$, if $k = \hat{z}_\alpha + 2 \cdot \hat{z}_\beta$ is the selected row, then $\mathcal{P}_B$ can compute

$$\lambda_{B,\gamma,k} = \lambda'_{B,\gamma} \oplus \lambda_{B,\gamma} \oplus \hat{z}_\alpha \cdot \lambda_{B,\beta} \oplus \hat{z}_\beta \cdot \lambda_{B,\alpha} \oplus \hat{z}_\alpha \cdot \hat{z}_\beta \quad (3)$$

and decrypt the corresponding row to obtain $\lambda_{A,\gamma,k}$. Taking the XOR of both shares yields

$$\begin{aligned}\lambda_{A,\gamma,k} \oplus \lambda_{B,\gamma,k} &= \lambda'_{A,\gamma} \oplus \lambda_{A,\gamma} \oplus \hat{z}_\alpha \cdot \lambda_{A,\beta} \oplus \hat{z}_\beta \cdot \lambda_{A,\alpha} \oplus \lambda'_{B,\gamma} \oplus \lambda_{B,\gamma} \oplus \hat{z}_\alpha \cdot \lambda_{B,\beta} \oplus \hat{z}_\beta \cdot \lambda_{B,\alpha} \oplus \hat{z}_\alpha \cdot \hat{z}_\beta \\ &= \underbrace{\lambda'_{A,\gamma} \oplus \lambda'_{B,\gamma}}_{\lambda_\alpha \cdot \lambda_\beta} \oplus \lambda_\gamma \oplus \hat{z}_\alpha \cdot \lambda_\beta \oplus \hat{z}_\beta \cdot \lambda_\alpha \oplus \hat{z}_\alpha \cdot \hat{z}_\beta \\ &= \lambda_\alpha \cdot \lambda_\beta \oplus \lambda_\gamma \oplus \hat{z}_\alpha \cdot \lambda_\beta \oplus \hat{z}_\beta \cdot \lambda_\alpha \oplus \hat{z}_\alpha \cdot \hat{z}_\beta \\ &= (\lambda_\alpha \oplus \hat{z}_\alpha) \cdot (\lambda_\beta \oplus \hat{z}_\beta) \oplus \lambda_\gamma \\ &= z_\alpha \cdot z_\beta \oplus \lambda_\gamma\end{aligned}$$

which is exactly the masked output value.

Authenticating shared random masks. Noted by [34], $\mathcal{P}_A$ can violate the correctness of the garbled table by changing the orders of rows or encrypting incorrect rows. To cope with this issue, [34] enforces every shared random mask from each party to be authenticated by using subfield VOLE. In particular, for each $p \in \{A, B\}$, let $\mathcal{P}_p$ keep $\Delta_p \in \mathbb{F}_{2^\rho}$ for some security parameter ρ . Then, for each $u \in \{\lambda_{p,w}\}_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W} \cup \{\lambda'_{p,w}\}_{w \in \mathcal{I}_W}$ held by $\mathcal{P}_p$, $\mathcal{P}_p$ and $\mathcal{P}_{\bar{p}}$ respectively hold $(u, M[u])$ and $K[u]$ s.t.

$$M[u] = K[u] \oplus u \cdot \Delta_{\bar{p}}.$$

Generating $M[u]$ and $K[u]$ can be done by using functionality $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$, recalled in Figure 4.

Let ρ be the statistical security parameter. $\mathcal{P}_A$ and $\mathcal{P}_B$ run as follows.

1. Assuming no Δ_A, Δ_B stored, upon receiving Δ_A from $\mathcal{P}_A$ and (init) from $\mathcal{P}_B$, sample $\Delta_B \xleftarrow{\$} \{0, 1\}^\rho$, and store Δ_A and Δ_B . Then, send Δ_B to $\mathcal{P}_B$.
2. Upon receiving (random, $r_A, M[r_A], K[r_B]$) from $\mathcal{P}_A$ and (random) from $\mathcal{P}_B$, sample $r_B \xleftarrow{\$} \{0, 1\}$, and compute $M[r_B]$ and $K[r_A]$ s.t. $M[r_p] = K[r_p] \oplus r_p \cdot \Delta_{\bar{p}}$ for $p \in \{A, B\}$. Send $(r_B, M[r_B], K[r_A])$ to $\mathcal{P}_B$.
3. Upon receiving (and, $(\lambda_{A,i}, M[\lambda_{A,i}], K[\lambda_{B,i}])_{i=1}^3$) from $\mathcal{P}_A$ and (and, $(\lambda_{B,i}, M[\lambda_{B,i}], K[\lambda_{A,i}])_{i=1}^2$) from $\mathcal{P}_B$, send (cheat) to $\mathcal{P}_B$ if it does not hold that $M[\lambda_{p,i}] = K[\lambda_{p,i}] \oplus \lambda_{p,i} \cdot \Delta_{\bar{p}} \forall p \in \{A, B\}, \forall i \in \{1, 2\}$. Otherwise, compute $\lambda_{B,3} = ((\lambda_{A,1} \oplus \lambda_{B,1}) \cdot (\lambda_{A,2} \oplus \lambda_{B,2})) \oplus \lambda_{A,3}$ and $M[\lambda_{B,3}]$ and $K[\lambda_{A,3}]$ s.t. $M[\lambda_{p,3}] = K[\lambda_{p,3}] \oplus \lambda_{p,3} \cdot \Delta_{\bar{p}} \forall p \in \{A, B\}$. Send $(\lambda_{B,3}, M[\lambda_{B,3}], K[\lambda_{A,3}])$ to $\mathcal{P}_B$.

Fig. 4. Functionality $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ [34, Figure 1].

3.2 Making WRK Publicly Auditable

Our purpose is to make the WRK protocol publicly auditable. That is, a public verifier can trust that the output from $\mathcal{P}_B$ obtained after the garbling, by $\mathcal{P}_A$, and evaluating, by $\mathcal{P}_B$, is correct. We follow the recent VOLEitH approach [6,17,13] (c.f. Section 2) by applying SoftSpokenOT [32] for making VOLE-based protocols (e.g., Wolverine [35] and QuickSilver [37]) publicly verifiable as follows.

Parties. Besides $\mathcal{P}_A$ and $\mathcal{P}_B$, we additionally have another public party called the verifier $\mathcal{V}$ for verifying the transcript communicated between $\mathcal{P}_A$ and $\mathcal{P}_B$.

Obstacles. Assume that we transform all authenticated randomness of $\mathcal{P}_A$ and $\mathcal{P}_B$ into the forms specified by SoftSpokenOT (c.f. Section 2). This transformation requires making Δ_A and Δ_B public. Naively doing so introduces the following obstacles.

Leaking both labels. In WRK protocol, beside using Δ_p for authenticating $\mathcal{P}_p$'s secrets, computing garbled tables requires $\mathcal{P}_A$ to additionally sample the labels $L_{w,0}$ and $L_{w,1}$ corresponding to the assumptions $\hat{z}_w = 0$ and $\hat{z}_w = 1$, respectively, for every wire w . To be compatible with free XORs (i.e., $\hat{z}_\gamma = \hat{z}_\alpha \oplus \hat{z}_\beta$ for any XOR gate $(\alpha, \beta, \gamma, \oplus)$), [34] enforces $L_{w,0} \oplus \Delta_A = L_{w,1}$ with Δ_A only known to $\mathcal{P}_A$. Consequently, $\mathcal{P}_B$ learns only one of $L_{w,0}$ or $L_{w,1}$, for each wire w . When making public audibility, if Δ_A is public, knowing either $L_{w,0}$ or $L_{w,1}$ implies knowledge of both labels, hence violating the security of WRK protocol.

Verifying shared AND triples. For verifying each AND gate $(\alpha, \beta, \gamma, \wedge)$, $\mathcal{V}$ should be able to verify that $(\underbrace{\lambda_{A,\alpha} \oplus \lambda_{B,\alpha}}_{\lambda_\alpha}) \cdot$

$(\underbrace{\lambda_{A,\beta} \oplus \lambda_{B,\beta}}_{\lambda_\beta}) = \underbrace{\lambda'_{A,\gamma} \oplus \lambda'_{B,\gamma}}_{\lambda_\alpha \cdot \lambda_\beta}$. Here, in WRK protocol, $\mathcal{P}_A$ and $\mathcal{P}_B$ rely on the outputs of $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$, guaranteeing

the above identity holds. However, when making public audibility, since $\mathcal{V}$ does not receive those outputs from $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$, $\mathcal{V}$ cannot verify whether such multiplications hold.

Our approach. We cope with the above-mentioned issues as follows.

Authenticating with $\mathcal{F}_{\text{VOLE}}^{\mathcal{P}, \mathcal{V}, \cdot}$ (c.f. Figure 3). Note that Δ_A employed in WRK protocol is (i) authenticating $\mathcal{P}_B$'s shared random masks and (ii) allowing $\mathcal{P}_B$ to compute $L_{\gamma, \hat{z}_\gamma}$ for any $(\alpha, \beta, \gamma, \wedge)$. As discussed above, making Δ_A public violates $\mathcal{P}_A$'s privacy. Hence, in our design, we still keep both Δ_A and Δ_B private. Besides, to enable public audibility, we instead make VOLEitH correlations for them as follows. For each wire w and each $p \in \{A, B\}$, the secret $\lambda_{p,w}$ of $\mathcal{P}_p$ is authenticated via VOLEitH correlations as $M'_j[\lambda_{p,w}] = K'_j[\lambda_{p,w}] \oplus \lambda_{p,w} \cdot \nabla_{\bar{p},j} \forall j \in [\kappa]$ where κ is the number of repetitions for amplifying soundness, $(\nabla_{\bar{p},j})_{j=1}^\kappa$ is publicly determined by $\mathcal{V}$, and $(M'_j[\lambda_{p,w}])_{j=1}^\kappa$ and $(K'_j[\lambda_{p,w}])_{j=1}^\kappa$ are the sequences of VOLEitH tags and keys for VOLEitH correlations. Since $\nabla_{A,j}$ is not employed to determine $L_{\gamma, \hat{z}_\gamma}$ for any AND gate $(\alpha, \beta, \gamma, \wedge)$, $\mathcal{P}_A$'s privacy is guaranteed as long as Δ_A is not compromised.

Handling shared XORs. For each XOR gate $(\alpha, \beta, \gamma, \oplus)$, since VOLEitH correlations are “homomorphic” w.r.t. XOR, $\mathcal{P}_p$, for $p \in \{A, B\}$, can locally evaluate by computing $\lambda_{p,\gamma} := \lambda_{p,\alpha} \oplus \lambda_{p,\beta}$ and $M'_j[\lambda_{p,\gamma}] := M'_j[\lambda_{p,\alpha}] \oplus M'_j[\lambda_{p,\beta}]$ for $j \in [\kappa]$. When knowing the VOLEitH keys, $\mathcal{V}$ can homomorphically compute the keys, e.g., $K'_j[\lambda_{p,\gamma}] := K'_j[\lambda_{p,\alpha}] \oplus K'_j[\lambda_{p,\beta}]$, w.r.t. XOR gates, accordingly.

Proving shared AND triples. Each $\lambda'_{p,\gamma}$ must also be authenticated via VOLEitH accordingly w.r.t. $(\nabla_{\bar{p},j})_{j \in [\kappa]}$. For each AND gate $(\alpha, \beta, \gamma, \wedge)$, the parties must show that

$$(\underbrace{\lambda_{A,\alpha} \oplus \lambda_{B,\alpha}}_{\lambda_\alpha}) \cdot (\underbrace{\lambda_{A,\beta} \oplus \lambda_{B,\beta}}_{\lambda_\beta}) = \underbrace{\lambda'_{A,\gamma} \oplus \lambda'_{B,\gamma}}_{\lambda_\alpha \cdot \lambda_\beta}. \quad (4)$$

This relation cannot be verified directly using the multiplication trick from QuickSilver [37] because, for each $j \in [\kappa]$, we have $\lambda_{p,\alpha}$, $\lambda_{p,\beta}$, and $\lambda'_{p,\gamma}$ are authenticated via $\nabla_{\bar{p},j}$. It follows that this approach would require multiplying together the secrets of $\mathcal{P}_A$ and $\mathcal{P}_B$, e.g., the VOLEitH tags, leading to compromising their privacy. To this end, we apply Beaver's multiplication trick and the balls-and-bins method as follows.

Applying Beaver's multiplication trick [12]. We design $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$ (c.f. Figure 5), where $W = |\mathcal{I}_W|$, based on Beaver's multiplication trick (c.f. Section 2) for W AND gates, where $W \in \mathbb{N}$, and κ repetitions (discussed above for amplifying soundness purposes) as follows: Let $(x_p^{(i)}, y_p^{(i)}, z_p^{(i)})_{i=1}^W$ be held by $\mathcal{P}_p$, for $p \in \mathcal{P}$, and authenticated via $(\nabla_{\bar{p},j})_{j \in [\kappa]}$. Via $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$, our purpose is to show that

$$(x_A^{(i)} \oplus x_B^{(i)}) \cdot (y_A^{(i)} \oplus y_B^{(i)}) = z_A^{(i)} \oplus z_B^{(i)} \forall i \in [W]. \quad (5)$$

To this end, we assume that each $\mathcal{P}_p$ additionally keeps $(a_{p,j}^{(i)}, b_{p,j}^{(i)}, c_{p,j}^{(i)})_{j \in [\kappa], i \in [W]}$, authenticated via $(\nabla_{\bar{p},j})_{j=1}^\kappa$ such that

$$(a_{A,j}^{(i)} \oplus a_{B,j}^{(i)}) \cdot (b_{A,j}^{(i)} \oplus b_{B,j}^{(i)}) = c_{A,j}^{(i)} \oplus c_{B,j}^{(i)} \quad \forall j \in [\kappa], \forall i \in [W]. \quad (6)$$

Then, we can apply the Beaver's multiplication trick adapted for VOLEitH (c.f. Section 2) to guarantee that, for each $i \in [W]$, (5) holds by using authenticated $x_p^{(i)}$, $y_p^{(i)}$, and $z_p^{(i)}$, for $p \in \{A, B\}$. The verifier $\mathcal{V}$ can check the authenticity of revealed values from $\mathcal{P}_A$ and $\mathcal{P}_B$ via VOLEitH correlations and whether $\tilde{z}_j^{(i)} = d_j^{(i)} \cdot e_j^{(i)}$ similar to that in Section 2. Nevertheless, we assumed that (6) holds but $\mathcal{V}$ is unable to verify it.

Using balls-and-bins method [3,35]. In order for $\mathcal{P}_A$ and $\mathcal{P}_B$ to achieve such triples satisfying (6), we follow the balls-and-bins method: In our protocol, there are W such AND gates. To check all W AND gates, we generate L pairs of shared random AND triples. Then, we apply the balls-and-bins method [3,35] (recalled in Lemma 1 below) to check all W AND gates. By appropriately setting parameters (say, L , bs , and rm), the probability stated at the end of Lemma 1 can become negligible. Therefore, assume that such a probability is negligible.

Lemma 1 (Balls-and-bins [3,35]). *There are $L = W \cdot \text{bs} + \text{rm}$ balls, where $W, \text{bs}, \text{rm} \in \mathbb{N}$, labeled either good or bad. (1) rm random balls are chosen. (2) The remaining $W \cdot \text{bs}$ are randomly partitioned into W bins where each bin contains exactly bs balls. A bin is fully good (respectively, fully bad) if all balls in this bin are good (respectively, bad). Then, the probability that (i) the first randomly chosen rm random balls are labeled good, and (ii) at least one bin is fully bad and all other bins are either fully good or fully bad is at most $\left(\frac{L}{\text{bs}}\right)^{-1}$.*

This protocol is run between $\mathcal{P}_A$, $\mathcal{P}_B$, and $\mathcal{V}$. We assume that authenticated $\mathcal{P}_A$'s values (respectively, $\mathcal{P}_B$'s values) are w.r.t. $\nabla_{B,j}$ (respectively, $\nabla_{A,j}$) for $j \in [\kappa]$ and $\kappa, W \in \mathbb{N}$. When writing j and i as subscripts of sets/sequences or after symbol $\forall$ without specified ranges, it means $j \in [\kappa]$ and $i \in [W]$, respectively.

1. For $p \in \{A, B\}$, $\mathcal{P}_p$ keeps $(x_p^{(i)}, y_p^{(i)}, z_p^{(i)})_i$, $(M'_j[x_p^{(i)}], M'_j[y_p^{(i)}], M'_j[z_p^{(i)}])_{j,i}$, $(a_{p,j}^{(i)}, b_{p,j}^{(i)}, c_{p,j}^{(i)})$, $M'_j[a_{p,j}^{(i)}], M'_j[b_{p,j}^{(i)}], M'_j[c_{p,j}^{(i)}]_{j,i}$. $\mathcal{P}_A$ and $\mathcal{P}_B$ work as follows.

- For $p \in \{A, B\}$, $\mathcal{P}_p$ publishes $d_{p,j}^{(i)} := x_p^{(i)} \oplus a_{p,j}^{(i)}$, $e_{p,j}^{(i)} := y_p^{(i)} \oplus b_{p,j}^{(i)}$, $M'_j[d_{p,j}^{(i)}] := M'_j[x_p^{(i)}] \oplus M'_j[a_{p,j}^{(i)}]$, and $M'_j[e_{p,j}^{(i)}] := M'_j[y_p^{(i)}] \oplus M'_j[b_{p,j}^{(i)}] \quad \forall j, \forall i$.
- Each party locally computes $d_j^{(i)} := d_{A,j}^{(i)} \oplus d_{B,j}^{(i)}$ and $e_j^{(i)} := e_{A,j}^{(i)} \oplus e_{B,j}^{(i)} \quad \forall j, \forall i$.
- For $p \in \{A, B\}$, $\mathcal{P}_p$ publishes $\tilde{z}_{p,j}^{(i)} := z_p^{(i)} \oplus c_{p,j}^{(i)} \oplus d_j^{(i)} \cdot b_{p,j}^{(i)} \oplus e_j^{(i)} \cdot a_{p,j}^{(i)}$ and $M'_j[\tilde{z}_{p,j}^{(i)}] := M'_j[z_p^{(i)}] \oplus M'_j[c_{p,j}^{(i)}] \oplus d_j^{(i)} \cdot M'_j[b_{p,j}^{(i)}] \oplus e_j^{(i)} \cdot M'_j[a_{p,j}^{(i)}] \quad \forall j, \forall i$.
- Each party checks whether $\tilde{z}_{A,j}^{(i)} \oplus \tilde{z}_{B,j}^{(i)} \oplus d_j^{(i)} \cdot e_j^{(i)} = 0 \quad \forall j, \forall i$.

2. For $p \in \{A, B\}$, on common inputs $(\nabla_{\bar{p},j})_j$, $(K'_j[x_p^{(i)}], K'_j[y_p^{(i)}], K'_j[z_p^{(i)}])_{j,i}$, $(K'_j[a_{p,j}^{(i)}], K'_j[b_{p,j}^{(i)}], K'_j[c_{p,j}^{(i)}])_{j,i}$, $\mathcal{V}$ can compute and check as follows.

- $\mathcal{V}$ computes $K'_j[d_{p,j}^{(i)}] := K'_j[x_p^{(i)}] \oplus K'_j[a_{p,j}^{(i)}]$, $K'_j[e_{p,j}^{(i)}] := K'_j[y_p^{(i)}] \oplus K'_j[b_{p,j}^{(i)}]$ and checks whether $M'_j[d_{p,j}^{(i)}] = K'_j[d_{p,j}^{(i)}] \oplus d_{p,j}^{(i)} \cdot \nabla_{\bar{p},j}$, $M'_j[e_{p,j}^{(i)}] = K'_j[e_{p,j}^{(i)}] \oplus e_{p,j}^{(i)} \cdot \nabla_{\bar{p},j} \quad \forall j, \forall i$.
- $\mathcal{V}$ computes $K'_j[\tilde{z}_{p,j}^{(i)}] := K'_j[z_p^{(i)}] \oplus K'_j[c_{p,j}^{(i)}] \oplus d_j^{(i)} \cdot K'_j[b_{p,j}^{(i)}] \oplus e_j^{(i)} \cdot K'_j[a_{p,j}^{(i)}]$ and checks whether $M'_j[\tilde{z}_{p,j}^{(i)}] = K'_j[\tilde{z}_{p,j}^{(i)}] \oplus \tilde{z}_{p,j}^{(i)} \cdot \nabla_{\bar{p},j} \quad \forall j, \forall i$.

Fig. 5. Sub-Protocol $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$. Those highlighted are made public for auditability.

For each repetition $j \in [\kappa]$, assume that each $\mathcal{P}_p$ keeps $(\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{\ell \in [L]}$ such that $(\tilde{a}_{A,j}^{(\ell)} \oplus \tilde{a}_{B,j}^{(\ell)}) \cdot (\tilde{b}_{A,j}^{(\ell)} \oplus \tilde{b}_{B,j}^{(\ell)}) = \tilde{c}_{A,j}^{(\ell)} \oplus \tilde{c}_{B,j}^{(\ell)}$ for $\ell \in [L]$. These values are also authenticated by $\nabla_{B,j}$ and $\nabla_{A,j}$ accordingly. Although $(\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{\ell \in [L]}$ is authenticated via $\nabla_{\bar{p},j}$, we are not guaranteed that $(\tilde{a}_{A,j}^{(\ell)} \oplus \tilde{a}_{B,j}^{(\ell)}) \cdot (\tilde{b}_{A,j}^{(\ell)} \oplus \tilde{b}_{B,j}^{(\ell)}) = \tilde{c}_{A,j}^{(\ell)} \oplus \tilde{c}_{B,j}^{(\ell)}$ $\ell \in [L]$. From Lemma 1, we have balls labeled either good or bad. When considering in our context, the ball $\ell \in [L]$ is understood to be good if it holds that $(\tilde{a}_{A,j}^{(\ell)} \oplus \tilde{a}_{B,j}^{(\ell)}) \cdot (\tilde{b}_{A,j}^{(\ell)} \oplus \tilde{b}_{B,j}^{(\ell)}) = \tilde{c}_{A,j}^{(\ell)} \oplus \tilde{c}_{B,j}^{(\ell)}$, and bad otherwise. Hence, from Lemma 1, we first randomly pick rm random balls to check whether they are all labeled good by forcing both $\mathcal{P}_A$ and $\mathcal{P}_B$ to reveal all values as well as the VOLEitH tags corresponding to the chosen rm balls that we just mentioned, i.e., if the ball ℓ is chosen among rm balls, $\tilde{a}_{p,j}^{(\ell)}$, $\tilde{b}_{p,j}^{(\ell)}$, and $\tilde{c}_{p,j}^{(\ell)}$ for $p \in \{A, B\}$ as well as their corresponding VOLEitH tags w.r.t. $\nabla_{\bar{p},j}$. For the remaining $W \cdot \text{bs}$ balls, we randomly partition them into W bins, corresponding to W AND gates, such that each bin contains exactly bs balls. Then, we run bs times s.t.,

for $k \in [\text{bs}]$ and $i \in [W]$, we use the k -th ball from bin i to test the i -th AND gate by employing protocol $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$ in Figure 5. Notice that if secret values of $\mathcal{P}_A$ and $\mathcal{P}_B$ do not hold at some AND gate, then, to pass the test by $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$, all balls in the corresponding bin to such AND must be labeled **bad**, since any good ball will make the test failed. It follows from Lemma 1 that such a case can happen with negligible probability with appropriate settings of rm and bs . Hence, we follow [35] to choose and partition the balls as follows. Let $\mathcal{V}$ generate the permutation $\pi_j : [L] \rightarrow [L]$ and send it to both $\mathcal{P}_A$ and $\mathcal{P}_B$. Then, for $p \in \{A, B\}$, $\mathcal{P}_p$ shuffles $(\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{\ell \in [L]}$ by assigning the tuple

$$(a_{p,j}^{(\ell)}, b_{p,j}^{(\ell)}, c_{p,j}^{(\ell)}) := (\tilde{a}_{p,j}^{(\pi_j(\ell))}, \tilde{b}_{p,j}^{(\pi_j(\ell))}, \tilde{c}_{p,j}^{(\pi_j(\ell))}) \quad \forall \ell \in [L]$$

as well as the corresponding VOLEitH tags w.r.t. $\nabla_{\bar{p},j}$. Notice that we do not need to shuffle the keys at the moment as we are employing SoftSpokenOT. Then, both $\mathcal{P}_A$ and $\mathcal{P}_B$ reveal $(a_{p,j}^{(i)}, b_{p,j}^{(i)}, c_{p,j}^{(i)})_{p \in \{A,B\}, i \in [W \cdot \text{bs} + 1, W \cdot \text{bs} + \text{rm}]}$, and their corresponding VOLEitH tags w.r.t. $\nabla_{A,j}$ and $\nabla_{B,j}$ for testing rm balls. For $k \in [\text{bs}]$, we use the sequence $(a_{p,j}^{(i)}, b_{p,j}^{(i)}, c_{p,j}^{(i)})_{p \in \{A,B\}, i \in [(k-1) \cdot W + 1, k \cdot W]}$ to test the W AND gates by running $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$ for all κ repetitions. **Generating VOLEitH Correlations.** Since from above we requires authenticating the secret bits $\mathcal{P}_A$ and $\mathcal{P}_B$ via VOLEitH correlation, to simplify the design of our publicly auditable garbled circuit, we define $\mathcal{F}_{\text{sVOLE-2PC}}$ and its realization $\Pi_{\text{sVOLE-2PC}}$ in Section 4 for generating VOLEitH tags and keys for

$$(\lambda_{p,w})_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (\lambda'_{p,w})_{w \in \mathcal{I}_W}, \text{ and } (\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{\ell \in [L]} \text{ of } \mathcal{P}_p \text{ for } p \in \{A, B\}.$$

Then, in Section 5, we design our publicly auditable garbled circuit by designing protocol $\Pi_{\text{pa-2PC}}$ by combining protocols $\Pi_{\text{pa-2PC-pre}}$, $\Pi_{\text{pa-2PC-eval}}$, and $\Pi_{\text{pa-2PC-ver}}$ in Figures 9, 10, and 11, respectively. The design of $\Pi_{\text{pa-2PC}}$ follows the above technical overview that we just discussed.

4 $\mathcal{F}_{\text{sVOLE-2PC}}$ and $\Pi_{\text{sVOLE-2PC}}$

Let us first define $\mathcal{F}_{\text{sVOLE-2PC}}$ in Figure 6 as a subroutine for generating suitable VOLEitH tags and keys. $\mathcal{F}_{\text{sVOLE-2PC}}$ has three parties: $\mathcal{P}_A$, $\mathcal{P}_B$, and a verifier $\mathcal{V}$.

4.1 Functionality $\mathcal{F}_{\text{sVOLE-2PC}}$

Following the discussion in Section 3.2, $\mathcal{V}$ plays the role of a public party to verify the transcript, including those exchanged between $\mathcal{P}_A$ and $\mathcal{P}_B$. This functionality is compatible with VOLEitH [6]. Specifically, for a function f represented by a Boolean circuit $\mathcal{C}$ with $\text{spec}(\mathcal{C}) = (\mathcal{I}_A, \mathcal{I}_B, \mathcal{I}_W, \mathcal{I}_O)$, we need the tuple $((\lambda_{p,w})_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (\lambda'_{p,w})_{w \in \mathcal{I}_W})$ kept by $\mathcal{P}_p$ to be authenticated via $(\nabla_{\bar{p},j})_{j \in [\kappa]}$. Moreover, as discussed in Section 3.2, for verifying AND gates by using balls-and-bins method (c.f. Lemma 1), the sequences $(\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{j \in [\kappa], \ell \in [L]}$ kept by $\mathcal{P}_p$ also need to be authenticated via $(\nabla_{\bar{p},j})_{j \in [\kappa]}$. Therefore, we define $\mathcal{F}_{\text{sVOLE-2PC}}$ for generating the global values $\nabla_{A,j}, \nabla_{B,j}$ for $j \in [\kappa]$. Then, it generates the VOLEitH tags and keys for authenticating by the above values w.r.t. $(\nabla_{B,j})_{j \in [\kappa]}$ and $(\nabla_{A,j})_{j \in [\kappa]}$ as specified in Figure 6.

4.2 Protocol $\Pi_{\text{sVOLE-2PC}}$

To realize $\mathcal{F}_{\text{sVOLE-2PC}}$, we use $\mathcal{F}_{\text{sVOLE}}$ (c.f. Section 2.2 and Figure 3). For a circuit that $\mathcal{P}_A$ and $\mathcal{P}_B$ interact to compute, recall that $\mathcal{I}_A$ and $\mathcal{I}_B$ are the sets of input wires that $\mathcal{P}_A$ and $\mathcal{P}_B$, respectively, know the actual values. $\mathcal{I}_W$ is the set of output wires of the AND gates.

For authenticating $\mathcal{P}_p$'s secrets, we let $\mathcal{P}_p$ and $\mathcal{V}$ access to $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}_p, \mathcal{V}, \tau, N}$, by sending (init), where τ is the bit size for the VOLEitH tags and keys, $N = |\mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W| + W + 3L$ and $W = |\mathcal{I}_W|$. Then, in the j -th invocation for $j \in [\kappa]$, $\mathcal{P}_p$ receives as secret inputs random vectors $\mathbf{u}_j \in \mathbb{F}_2^N$ and $\mathbf{v}_j \in \mathbb{F}_2^N$ s.t.

$$\begin{aligned} \mathbf{u}_{p,j} &= \left((\bar{\lambda}_{p,w,j})_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W} \| (\bar{\lambda}'_{p,w,j})_{w \in \mathcal{I}_W} \| (\bar{a}_{p,j}^{(\ell)}, \bar{b}_{p,j}^{(\ell)}, \bar{c}_{p,j}^{(\ell)})_{\ell \in [L]} \right) \text{ and} \\ \mathbf{v}_{p,j} &= \left((\mathbf{M}'_j[\bar{\lambda}_{p,w,j}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W} \| (\mathbf{M}'_j[\bar{\lambda}'_{p,w,j}])_{w \in \mathcal{I}_W} \| (\mathbf{M}'_j[\bar{a}_{p,j}^{(\ell)}], \mathbf{M}'_j[\bar{b}_{p,j}^{(\ell)}], \mathbf{M}'_j[\bar{c}_{p,j}^{(\ell)}])_{\ell} \right). \end{aligned}$$

In other words, $\mathbf{v}_{p,j}$ contains VOLEitH tags corresponding to those in $\mathbf{u}_{p,j}$. Then, later on, $\mathcal{P}_p$ and $\mathcal{V}$ can send (get) to $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}_p, \mathcal{V}, \tau, N}$ to achieve the corresponding global value $(\nabla_{\bar{p},j})_{j=1}^{\kappa}$ and the corresponding public IT-MAC keys

$$\mathbf{w}_{p,j} = \left((\mathbf{K}'_j[\bar{\lambda}_{p,w,j}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (\mathbf{K}'_j[\bar{\lambda}'_{p,w,j}])_{w \in \mathcal{I}_W}, (\mathbf{K}'_j[\bar{a}_{p,j}^{(\ell)}], \mathbf{K}'_j[\bar{b}_{p,j}^{(\ell)}], \mathbf{K}'_j[\bar{c}_{p,j}^{(\ell)}])_{\ell} \right).$$

$\mathcal{F}_{\text{sVOLE-2PC}}$ is run by $\mathcal{P}_A$, $\mathcal{P}_B$, and a verifier $\mathcal{V}$ with the sets $\mathcal{I}_A, \mathcal{I}_B$, and $\mathcal{I}_W$, parameters κ, τ , and L . When denoting j, i , and ℓ as subscripts of sets/sequences or after symbol $\forall$ without specified ranges, it means $j \in [\kappa]$, $i \in [W]$, and $\ell \in [L]$, respectively.

1. On input $(\text{init}, (\lambda_{p,w})_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (\lambda'_{p,w})_{w \in \mathcal{I}_W}, (\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{j \in [\kappa]})$ from each $\mathcal{P}_p$,
 - (a) Sample $\nabla_{A,j}, \nabla_{B,j} \xleftarrow{\$} \mathbb{F}_{2^\tau} \forall j$. If $\mathcal{V}$ is corrupted, receive $(\nabla_{A,j}, \nabla_{B,j})_j$ from $\mathcal{V}$.
 - (b) For $p \in \{A, B\}$, if $\mathcal{P}_p$ is not corrupted, $\forall j$, sample $M'_j[\lambda_{p,w}] \xleftarrow{\$} \mathbb{F}_{2^\tau} \forall w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W$, $M'_j[\lambda'_{p,w}] \xleftarrow{\$} \mathbb{F}_{2^\tau} \forall w \in \mathcal{I}_W$, and $M'_j[\tilde{a}_{p,j}^{(\ell)}], M'_j[\tilde{b}_{p,j}^{(\ell)}], M'_j[\tilde{c}_{p,j}^{(\ell)}] \xleftarrow{\$} \mathbb{F}_{2^\tau} \forall \ell$. If $\mathcal{P}_p$ is corrupted, receive $((M'_j[\lambda_{p,w}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (M'_j[\lambda'_{p,w}])_{w \in \mathcal{I}_W}, (M'_j[\tilde{a}_{p,j}^{(\ell)}], M'_j[\tilde{b}_{p,j}^{(\ell)}], M'_j[\tilde{c}_{p,j}^{(\ell)}])_\ell)_j$ from $\mathcal{P}_p$. Then, compute $((K'_j[\lambda_{p,w}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (K'_j[\lambda'_{p,w}])_{w \in \mathcal{I}_W}, (K'_j[\tilde{a}_{p,j}^{(\ell)}], K'_j[\tilde{b}_{p,j}^{(\ell)}], K'_j[\tilde{c}_{p,j}^{(\ell)}])_\ell)_j$ s.t.

$$\begin{cases} M'_j[\lambda_{p,w}] = K'_j[\lambda_{p,w}] \oplus \lambda_{p,w} \cdot \nabla_{\bar{p},j} & \forall j \in [\kappa], \forall w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W, \\ M'_j[\lambda'_{p,w}] = K'_j[\lambda'_{p,w}] \oplus \lambda'_{p,w} \cdot \nabla_{\bar{p},j} & \forall j \in [\kappa], \forall w \in \mathcal{I}_W, \\ M'_j[\tilde{v}_{p,j}^{(\ell)}] = K'_j[\tilde{v}_{p,j}^{(\ell)}] \oplus \tilde{v}_{p,j}^{(\ell)} \cdot \nabla_{\bar{p},j} & \forall \tilde{v} \in \{\tilde{a}, \tilde{b}, \tilde{c}\}, \forall j \in [\kappa], \forall \ell \in [L]. \end{cases} \quad (7)$$
 - (c) If $\mathcal{V}$ is corrupted, receive $((K'_j[\lambda_{p,w}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (K'_j[\lambda'_{p,w}])_{w \in \mathcal{I}_W}, (K'_j[\tilde{a}_{p,j}^{(\ell)}], K'_j[\tilde{b}_{p,j}^{(\ell)}], K'_j[\tilde{c}_{p,j}^{(\ell)}])_\ell)_j$, for $p \in \{A, B\}$, from $\mathcal{V}$, and compute the following s.t. (7) holds.

$$((M'_j[\lambda_{p,w}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (M'_j[\lambda'_{p,w}])_{w \in \mathcal{I}_W}, (M'_j[\tilde{a}_{p,j}^{(\ell)}], M'_j[\tilde{b}_{p,j}^{(\ell)}], M'_j[\tilde{c}_{p,j}^{(\ell)}])_\ell)_j.$$
 - (d) For $p \in \{A, B\}$, send $((M'_j[\lambda_{p,w}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (M'_j[\lambda'_{p,w}])_{w \in \mathcal{I}_W}, (M'_j[\tilde{a}_{p,j}^{(\ell)}], M'_j[\tilde{b}_{p,j}^{(\ell)}], M'_j[\tilde{c}_{p,j}^{(\ell)}])_\ell)_j$ to $\mathcal{P}_p$.
2. On input (get) from $\mathcal{P}_A, \mathcal{P}_B$, and $\mathcal{V}$, $\mathcal{F}_{\text{sVOLE-2PC}}$ publishes $(\nabla_{A,j}, \nabla_{B,j})_{j \in [\kappa]}$, $((K'_j[\lambda_{p,w}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (K'_j[\lambda'_{p,w}])_{w \in \mathcal{I}_W}, (K'_j[\tilde{a}_{p,j}^{(\ell)}], K'_j[\tilde{b}_{p,j}^{(\ell)}], K'_j[\tilde{c}_{p,j}^{(\ell)}])_\ell)_j \forall p \in \{A, B\}$.

Fig. 6. Functionality $\mathcal{F}_{\text{sVOLE-2PC}}$. Those highlighted are made public for auditability.

Hence, the random secret values and VOLEitH tags (kept by $\mathcal{P}_p$), the public global value $\nabla_{\bar{p},j}$ and VOLEitH keys together form the VOLEitH correlations for the N bits of $\mathbf{u}_{p,j}$.

By using $\Pi_{\text{fix}}^{\mathcal{P}_p, \mathcal{V}}$ we can adapt the VOLEitH correlations, w.r.t. $\nabla_{\bar{p},j}$, for random values in vector $\mathbf{u}_{p,j}$ into those for

$$(\lambda_{p,w})_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (\lambda'_{p,w})_{w \in \mathcal{I}_W}, \text{ and } (\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{\ell \in [L]}$$

which are kept by $\mathcal{P}_p$.

Eventually, we describe $\Pi_{\text{sVOLE-2PC}}$ in Figure 7 in the $\mathcal{F}_{\text{sVOLE}}$ -hybrid model to realize $\mathcal{F}_{\text{sVOLE-2PC}}$ following the above discussion.

Security. We discuss the security of $\Pi_{\text{sVOLE-2PC}}$ in Theorem 1.

Theorem 1 (Security of $\Pi_{\text{sVOLE-2PC}}$). *The protocol $\Pi_{\text{sVOLE-2PC}}$ in Figure 7 securely realizes $\mathcal{F}_{\text{sVOLE-2PC}}$ in Figure 6 against malicious adversaries $\mathcal{P}_A$ and $\mathcal{P}_B$ with statistical security $2^{-\kappa \cdot \tau}$ in the $\mathcal{F}_{\text{sVOLE}}$ -hybrid model.*

We provide the proof sketch in Appendix B.

Communication Cost of $\Pi_{\text{sVOLE-2PC}}$ (c.f. Figure 7). Notice that we need to run in total κ times the functionality $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}_p, \mathcal{V}, \tau, N}$ for each $p \in \{A, B\}$. When realizing this functionality with $\Pi_{\text{sVOLE}}^{\mathcal{P}_p, \mathcal{V}, \tau, N}$ (c.f. Figure 1) and vector commitments (discussed in Section 2.2), it requires κ vector commitments and openings, subsequently determining the VOLEitH keys of authenticated values. Recall the notation $\text{costvc}_{\text{total}}^{\tau, N}$ from Section 2.2. Hence, in total, it requires $\kappa \cdot (\text{costvc}_{\text{total}}^{\tau, N} + \tau)$ for the κ invocations to $\Pi_{\text{sVOLE}}^{\mathcal{P}_p, \mathcal{V}, \tau, N}$ for each $p \in \{A, B\}$. Using protocol $\Pi_{\text{fix}}^{\mathcal{P}_p, \mathcal{V}}$ (c.f. Section 2.2), for $\kappa \cdot N$ times costs additionally $\kappa \cdot N$ bits for each $p \in \{A, B\}$. Hence, the total bits of communication of $\Pi_{\text{sVOLE-2PC}}$ is $2 \cdot \kappa \cdot (\text{costvc}_{\text{total}}^{\tau, N} + \tau + N)$ for both $p = A$ and $p = B$.

5 Publicly Auditable 2PC

We present the publicly auditable 2PC protocol $\Pi_{\text{pa-2PC}}$, in Section 5.2, following the technical overview in Section 3.2. This protocol realizes the functionality $\mathcal{F}_{\text{pa-2pc}}$ that we describe in Section 5.1.

Protocol $\Pi_{\text{svOLE-2PC}}$ runs between $\mathcal{P}_A$, $\mathcal{P}_B$, and a verifier $\mathcal{V}$ with The sets $\mathcal{I}_A, \mathcal{I}_B$, and $\mathcal{I}_W$, parameters κ, τ, L , and $N = |\mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W| + W + 3L$ where $W = |\mathcal{I}_W|$. When denoting j, i , and ℓ as subscripts of sets/sequences or after symbol $\forall$ without specifying the ranges, we understand that $j \in [\kappa]$, $i \in [W]$, and $\ell \in [L]$, respectively.

1. On inputs $(\text{init}, (\lambda_{p,w})_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (\lambda'_{p,w})_{w \in \mathcal{I}_W}, (\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{j,\ell})$ from $\mathcal{P}_p$ for $p \in \{A, B\}$,
 - (a) $\forall p \in \{A, B\}$, $\mathcal{P}_p$ and $\mathcal{V}$ send (init) to κ instances of $\mathcal{F}_{\text{svOLE}}^{\mathcal{P}_p, \mathcal{V}, \tau, N}$.
 - (b) $\forall p \in \{A, B\}$, $\forall j \in [\kappa]$, $\mathcal{P}_p$ receives outputs $\mathbf{u}_{p,j} = ((\bar{\lambda}_{p,w,j})_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W} \| (\bar{\lambda}'_{p,w,j})_{w \in \mathcal{I}_W} \| (\bar{a}_{p,j}^{(\ell)}, \bar{b}_{p,j}^{(\ell)}, \bar{c}_{p,j}^{(\ell)})_\ell)$ and $\mathbf{v}_{p,j} = ((M'_j[\bar{\lambda}_{p,w,j}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W} \| (M'_j[\bar{\lambda}'_{p,w,j}])_{w \in \mathcal{I}_W} \| (M'_j[\bar{a}_{p,j}^{(\ell)}], M'_j[\bar{b}_{p,j}^{(\ell)}], M'_j[\bar{c}_{p,j}^{(\ell)}])_\ell)$ from the j -th instance of $\mathcal{F}_{\text{svOLE}}^{\mathcal{P}_p, \mathcal{V}, \tau, N}$.
 - (c) $\forall p \in \{A, B\}$ and $\forall w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W$, $\mathcal{P}_p$ uses $\Pi_{\text{fix}}^{\mathcal{P}_p, \mathcal{V}}$ to obtain $\lambda_{p,w}$ and $(M'_j[\lambda_{p,w}])_j$, from inputs $\lambda_{p,w}$ and $(\bar{\lambda}_{p,w,j}, M'_j[\bar{\lambda}_{p,w,j}])_j$. Doing so requires $\hat{\lambda}_{p,w,j} := \bar{\lambda}_{p,w,j} \oplus \lambda_{p,w} \forall j, \forall w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W$ to be published.
 - (d) $\forall p \in \{A, B\}$ and $w \in \mathcal{I}_W$, $\mathcal{P}_p$ uses $\Pi_{\text{fix}}^{\mathcal{P}_p, \mathcal{V}}$ to obtain $\lambda'_{p,w}$ and $(M'_j[\lambda'_{p,w}])_j$, from inputs $\lambda'_{p,w}$ and $(\bar{\lambda}'_{p,w,j}, M'_j[\bar{\lambda}'_{p,w,j}])_j$. Doing so requires $\hat{\lambda}'_{p,w,j} := \bar{\lambda}'_{p,w,j} \oplus \lambda'_{p,w} \forall j, \forall w \in \mathcal{I}_W$ to be published.
 - (e) $\forall p \in \{A, B\}$ and $\forall \ell$, $\mathcal{P}_p$ uses $\Pi_{\text{fix}}^{\mathcal{P}_p, \mathcal{V}}$ to achieve $(M'_j[\bar{a}_{p,j}^{(\ell)}], M'_j[\bar{b}_{p,j}^{(\ell)}], M'_j[\bar{c}_{p,j}^{(\ell)}])_j$. Doing this requires $\hat{a}_{p,j}^{(\ell)} := \bar{a}_{p,j}^{(\ell)} \oplus \tilde{a}_{p,j}^{(\ell)}$, $\hat{b}_{p,j}^{(\ell)} := \bar{b}_{p,j}^{(\ell)} \oplus \tilde{b}_{p,j}^{(\ell)}$, and $\hat{c}_{p,j}^{(\ell)} := \bar{c}_{p,j}^{(\ell)} \oplus \tilde{c}_{p,j}^{(\ell)} \forall p \in \{A, B\}, \forall j, \forall \ell$ to be published.
2. On input (get) from $\mathcal{P}_A$, $\mathcal{P}_B$, and $\mathcal{V}$, $\Pi_{\text{svOLE-2PC}}$ works as follows:
 - (a) $\forall p \in \{A, B\}$, $\mathcal{P}_p$ and $\mathcal{V}$ send (get) to κ instances of $\mathcal{F}_{\text{svOLE}}^{\mathcal{P}_p, \mathcal{V}, \tau, N}$ to receive $(\nabla_{\bar{p},j})_j$ and $((K'_j[\bar{\lambda}_{p,w,j}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (K'_j[\bar{\lambda}'_{p,w,j}])_{w \in \mathcal{I}_W}, (K'_j[\bar{a}_{p,j}^{(\ell)}], K'_j[\bar{b}_{p,j}^{(\ell)}], K'_j[\bar{c}_{p,j}^{(\ell)}])_\ell)_j$.
 - (b) Any party can compute locally, by applying protocol $\Pi_{\text{fix}} \forall j$, as follows. For $p \in \{A, B\}$,

$$\begin{aligned}
\forall w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W, K'_j[\lambda_{p,w,j}] &:= K'_j[\bar{\lambda}_{p,w,j}] \oplus \hat{\lambda}_{p,w,j} \cdot \nabla_{\bar{p},j}; \\
\forall w \in \mathcal{I}_W, K'_j[\lambda'_{p,w,j}] &:= K'_j[\bar{\lambda}'_{p,w,j}] \oplus \hat{\lambda}'_{p,w,j} \cdot \nabla_{\bar{p},j}; \\
\forall \ell \in [L], K'_j[\bar{a}_{p,j}^{(\ell)}] &:= K'_j[\bar{a}_{p,j}^{(\ell)}] \oplus \hat{a}_{p,j}^{(\ell)} \cdot \nabla_{\bar{p},j}; \\
\forall \ell \in [L], K'_j[\bar{b}_{p,j}^{(\ell)}] &:= K'_j[\bar{b}_{p,j}^{(\ell)}] \oplus \hat{b}_{p,j}^{(\ell)} \cdot \nabla_{\bar{p},j}; \\
\forall \ell \in [L], K'_j[\bar{c}_{p,j}^{(\ell)}] &:= K'_j[\bar{c}_{p,j}^{(\ell)}] \oplus \hat{c}_{p,j}^{(\ell)} \cdot \nabla_{\bar{p},j}.
\end{aligned}$$

Fig. 7. Protocol $\Pi_{\text{svOLE-2PC}}$. Those highlighted are made public for auditability.

5.1 Functionality $\mathcal{F}_{\text{pa-2pc}}$

In the functionality $\mathcal{F}_{\text{pa-2pc}}$ (c.f. Figure 8), there are two involved parties, namely, $\mathcal{P}_A$ and $\mathcal{P}_B$, for evaluating a garbled circuit, and an additional party, namely, verifier $\mathcal{V}$, for auditing the evaluations, conducted by $\mathcal{P}_A$ and $\mathcal{P}_B$, of a function f , represented by a Boolean circuit $\mathcal{C}$ whose specification $\text{spec}(\mathcal{C}) = (\mathcal{I}_A, \mathcal{I}_B, \mathcal{I}_W, \mathcal{I}_O)$. We assume the output bits of f , captured by the wires in $\mathcal{I}_O$, from the evaluations of $\mathcal{P}_A$ and $\mathcal{P}_B$ are public. Therefore, both parties keep the same set of actual values of those in the output set $\mathcal{I}_O$. The verifier $\mathcal{V}$ accepts if $\mathcal{V}$ believes that the evaluations, by both $\mathcal{P}_A$ and $\mathcal{P}_B$, are correct w.r.t. f .

$\mathcal{F}_{\text{pa-2pc}}$ is run between $\mathcal{P}_A$, $\mathcal{P}_B$, and a verifier $\mathcal{V}$. The common input is a circuit for a function $f : \{0, 1\}^{|\mathcal{I}_A|} \times \{0, 1\}^{|\mathcal{I}_B|} \rightarrow \{0, 1\}^{|\mathcal{I}_O|}$.

1. Receive as inputs (input, $(z_w)_{w \in \mathcal{I}_A}$, $(z_w)_{w \in \mathcal{I}_O}$) and (input, $(z_w)_{w \in \mathcal{I}_B}$, $(z_w)_{w \in \mathcal{I}_O}$) from $\mathcal{P}_A$ and $\mathcal{P}_B$, respectively.
2. Upon receiving (prove) from all three parties, check whether
 - $\mathcal{P}_A$ and $\mathcal{P}_B$ share the same $(z_w)_{w \in \mathcal{I}_O}$ from the inputs of both parties; and
 - $f((z_w)_{w \in \mathcal{I}_A} \| (z_w)_{w \in \mathcal{I}_B}) = (z_w)_{w \in \mathcal{I}_O}$.

Returns **true** to $\mathcal{V}$ if all constraints hold and **false** otherwise.

Fig. 8. Functionality $\mathcal{F}_{\text{pa-2pc}}$.

5.2 Protocol $\Pi_{\text{pa-2PC}}$

We present the sub-protocols $\Pi_{\text{pa-2PC-pre}}$ (for preprocessing), $\Pi_{\text{pa-2PC-eval}}$ (for evaluating) and $\Pi_{\text{pa-2PC-ver}}$ (for verifying) in Figures 9, 10, and 11, respectively. They employ $\mathcal{F}_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$ (c.f. Figure 5) as a subroutine in the $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ - and $\mathcal{F}_{\text{SVOLE-2PC}}$ -hybrid models. Denote by $\Pi_{\text{pa-2PC}}$ the concatenation of $\Pi_{\text{pa-2PC-pre}}$, $\Pi_{\text{pa-2PC-eval}}$, and $\Pi_{\text{pa-2PC-ver}}$, i.e., $\Pi_{\text{pa-2PC}} = (\Pi_{\text{pa-2PC-pre}}, \Pi_{\text{pa-2PC-eval}}, \Pi_{\text{pa-2PC-ver}})$. $\Pi_{\text{pa-2PC}}$ hence realizes $\mathcal{F}_{\text{pa-2pc}}$ described in Section 5.1. The description of $\Pi_{\text{pa-2PC-pre}}$ and $\Pi_{\text{pa-2PC-eval}}$ follows those discussed in Section 3.2.

Sub-protocol $\Pi_{\text{pa-2PC-pre}}$ (c.f. Figure 9). This is for both $\mathcal{P}_A$ and $\mathcal{P}_B$ to communicate in advance before knowing the actual values of the input wires.

Parties: $\mathcal{P}_A$, $\mathcal{P}_B$, and a verifier $\mathcal{V}$.

Inputs: Both parties $\mathcal{P}_A$ and $\mathcal{P}_B$ agree on a common circuit for function $f : \{0, 1\}^{|\mathcal{I}_A|} \times \{0, 1\}^{|\mathcal{I}_B|} \rightarrow \{0, 1\}^{|\mathcal{I}_O|}$ specified by the sets $\mathcal{I}_A$, $\mathcal{I}_B$, $\mathcal{I}_W$, and $\mathcal{I}_O$. $\mathcal{P}_A$ and $\mathcal{P}_B$ agree to a commitment scheme $\mathcal{COM}$ (see Section 2.4).

Remark: Denoting j , i , and ℓ as subscripts of sets/sequences or after $\forall$ without specifying the ranges means $j \in [\kappa]$, $i \in [W]$, and $\ell \in [L]$, respectively.

Function-independent preprocessing:

1. For $p \in \{A, B\}$, $\mathcal{P}_p$ sends (init) to $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ to receive $\Delta_p \in \mathbb{F}_{2^\rho}$.
2. For $w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W$, both $\mathcal{P}_A$ and $\mathcal{P}_B$ send (random) to $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$. In return, $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ sends $(\lambda_{p,w}, M[\lambda_{p,w}], K[\lambda_{\bar{p},w}]) \in \mathbb{F}_2 \times \mathbb{F}_{2^\rho} \times \mathbb{F}_{2^\rho}$ to $\mathcal{P}_p$ for $p \in \{A, B\}$. Define $\lambda_w := \lambda_{A,w} \oplus \lambda_{B,w}$. $\mathcal{P}_A$ picks a $\mathbf{L}_{w,0} \xleftarrow{\$} \mathbb{F}_{2^\rho}$.
3. $\mathcal{P}_A$ and $\mathcal{P}_B$ invoke $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ $L \cdot \kappa$ times to obtain $(\tilde{a}_{A,j}^{(\ell)}, \tilde{b}_{A,j}^{(\ell)}, \tilde{c}_{A,j}^{(\ell)})$ and $(\tilde{a}_{B,j}^{(\ell)}, \tilde{b}_{B,j}^{(\ell)}, \tilde{c}_{B,j}^{(\ell)})$, respectively, s.t. $(\tilde{a}_{A,j}^{(\ell)} \oplus \tilde{a}_{B,j}^{(\ell)}) \cdot (\tilde{b}_{A,j}^{(\ell)} \oplus \tilde{b}_{B,j}^{(\ell)}) = \tilde{c}_{A,j}^{(\ell)} \oplus \tilde{c}_{B,j}^{(\ell)} \forall \ell, \forall j$.

Function-dependent preprocessing:

4. Following the topological ordering of the circuit, $\mathcal{P}_A$ and $\mathcal{P}_B$ work as follows.

$\forall (\alpha, \beta, \gamma, \oplus), \forall p \in \{A, B\}$, $\mathcal{P}_p$ computes $(\lambda_{p,\gamma}, M[\lambda_{p,\gamma}], K[\lambda_{\bar{p},\gamma}]) := (\lambda_{p,\alpha} \oplus \lambda_{p,\beta}, M[\lambda_{p,\alpha}] \oplus M[\lambda_{p,\beta}], K[\lambda_{\bar{p},\alpha}] \oplus K[\lambda_{\bar{p},\beta}])$, and $\mathcal{P}_A$ additionally computes $\mathbf{L}_{\gamma,0} := \mathbf{L}_{\alpha,0} \oplus \mathbf{L}_{\beta,0}$.

$\forall (\alpha, \beta, \gamma, \wedge): \forall p \in \{A, B\}$,

 - (a) $\mathcal{P}_p$ sends (and, $(\lambda_{p,\alpha}, M[\lambda_{p,\alpha}], K[\lambda_{\bar{p},\alpha}]), (\lambda_{p,\beta}, M[\lambda_{p,\beta}], K[\lambda_{\bar{p},\beta}])$) to $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$. Then, $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ sends $(\lambda'_{p,\gamma}, M[\lambda'_{p,\gamma}], K[\lambda_{\bar{p},\gamma}])$ to $\mathcal{P}_p$ s.t. $\lambda'_{A,\gamma} \oplus \lambda_{B,\gamma} = \lambda_\alpha \cdot \lambda_\beta$.
 - (b) $\forall k \in [0, 3]$, let $(k_0, k_1) = \text{bin}_2(k)$, $u_A := 1$, and $u_B := 0$.
 $\mathcal{P}_p$ sets $\lambda_{p,\gamma,k} := \lambda'_{p,\gamma} \oplus \lambda_{p,\gamma} \oplus k_0 \cdot \lambda_{p,\beta} \oplus k_1 \cdot \lambda_{p,\alpha} \oplus u_p \cdot k_0 \cdot k_1$,
 $M[\lambda_{p,\gamma,k}] := M[\lambda'_{p,\gamma}] \oplus M[\lambda_{p,\gamma}] \oplus k_0 \cdot M[\lambda_{p,\beta}] \oplus k_1 \cdot M[\lambda_{p,\alpha}]$,
 $K[\lambda_{\bar{p},\gamma,k}] := K[\lambda_{\bar{p},\gamma}] \oplus K[\lambda_{\bar{p},\gamma}] \oplus k_0 \cdot K[\lambda_{\bar{p},\beta}] \oplus k_1 \cdot K[\lambda_{\bar{p},\alpha}] \oplus u_p \cdot k_0 \cdot k_1 \cdot \Delta_p$.
5. **Obtaining VOLEitH tags:** For $p \in \{A, B\}$, $\mathcal{P}_p$ sends (init, $(\lambda_{p,w})_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (\lambda'_{p,w})_{w \in \mathcal{I}_W}, (\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{j \in [\kappa], \ell \in [L]}$) to $\mathcal{F}_{\text{SVOLE-2PC}}$ (c.f. Figure 6).
 Then, $\mathcal{P}_p$ receives $(M'_j[\lambda_{p,w}])_{j \in [\kappa], w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (M'_j[\lambda'_{p,w}])_{j \in [\kappa], w \in \mathcal{I}_W}, (M'_j[\tilde{a}_{p,j}^{(\ell)}], M'_j[\tilde{b}_{p,j}^{(\ell)}], M'_j[\tilde{c}_{p,j}^{(\ell)}])_{j \in [\kappa], \ell \in [L]}$.
6. **Making VOLEitH proof and components for garbled tables:** For $p \in \{A, B\}$,
 - For each $(\alpha, \beta, \gamma, \oplus)$, $\mathcal{P}_p$ computes $M'_j[\lambda_{p,\gamma}] := M'_j[\lambda_{p,\alpha}] \oplus M'_j[\lambda_{p,\beta}] \forall j \in [\kappa]$.
 - For each $(\alpha, \beta, \gamma, \wedge)$, $\forall k \in [0, 3]$, let $(k_0, k_1) = \text{bin}_2(k)$. Then, $\mathcal{P}_p$ computes $M'_j[\lambda_{p,\gamma,k}] := M'_j[\lambda'_{p,\gamma}] \oplus M'_j[\lambda_{p,\gamma}] \oplus k_0 \cdot M'_j[\lambda_{p,\beta}] \oplus k_1 \cdot M'_j[\lambda_{p,\alpha}] \forall j \in [\kappa]$.
7. **Encrypting and committing:** For each $(\alpha, \beta, \gamma, \wedge)$,
 $\mathcal{P}_A$ computes $\mathbf{L}_{\alpha,1} := \mathbf{L}_{\alpha,0} \oplus \Delta_A$ and $\mathbf{L}_{\beta,1} := \mathbf{L}_{\beta,0} \oplus \Delta_A$.
 $\mathcal{P}_A$ publishes $G_{\gamma,k}$, $\forall k \in [0, 3]$ with $(k_0, k_1) = \text{bin}_2(k)$, where

$$G_{\gamma,k} := H(\mathbf{L}_{\alpha,k_0}, \mathbf{L}_{\beta,k_1}, \gamma, k) \oplus (\lambda_{A,\gamma,k}, M[\lambda_{A,\gamma,k}], (M'_j[\lambda_{A,\gamma,k}])_{j=1}^\kappa, \mathbf{L}_{\gamma,0} \oplus K[\lambda_{B,\gamma,k}] \oplus \lambda_{A,\gamma,k} \cdot \Delta_A).$$
 $\mathcal{P}_B$ samples randomness $\text{rand}_{\gamma,k} \forall k \in [0, 3]$ and commits to $(\lambda_{B,\gamma,k}, (M'_j[\lambda_{B,\gamma,k}])_j)$ via $\mathcal{COM}$, by using $\text{rand}_{\gamma,k}$, to obtain and publish commitment $\text{cMB}_{\gamma,k}$.

Fig. 9. Sub-protocol $\Pi_{\text{pa-2PC-pre}}$. Those highlighted are made public for auditability.

Constructing garbled tables. Following the technical overview (c.f. Section 3.2) and WRK protocol [34], the parties use $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ to generate components including $\Delta_p \in \mathbb{F}_{2^\rho}$ for some security parameter ρ , and the sequences of bits $(\lambda_{p,w})_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}$, $(\lambda'_{p,w})_{w \in \mathcal{I}_W}$, and $(\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{\ell \in [L], j \in [\kappa]}$ authenticated via $\Delta_{\bar{p}}$. Both parties also call $\mathcal{F}_{\text{SVOLE-2PC}}$ (c.f. Figure 6) to generate the VOLEitH tags for those components. Additionally, $\mathcal{P}_A$ computes the

labels $L_{w,0}$, for $w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W$. These labels are subsequently used by $\mathcal{P}_B$ to decrypt one row in each garbled table.

Following [34], $\mathcal{P}_A$ then proceeds to compute all the garbled tables w.r.t. AND gates. The computation requires the associated labels $L_{w,0}$ and $L_{w,1}$ corresponding to the assumptions $\hat{z}_w = 0$ and $\hat{z}_w = 1$, respectively. To be compatible with free XORs, [34] enforces $L_{w,0} \oplus \Delta_A = L_{w,1}$ where Δ_A is only known to $\mathcal{P}_A$. Consequently, $\mathcal{P}_B$ learns only one of $L_{w,0}$ or $L_{w,1}$ for each wire w .

- For each XOR gate $(\alpha, \beta, \gamma, \oplus)$, by enforcing $L_{\gamma,0} = L_{\alpha,0} \oplus L_{\beta,0}$, we can see that $L_{\gamma,\hat{z}_\gamma} = L_{\alpha,\hat{z}_\alpha} \oplus L_{\beta,\hat{z}_\beta}$.
- For each AND gate $(\alpha, \beta, \gamma, \wedge)$, $\mathcal{P}_A$ needs to compute a 4-row garbled table encrypting $\lambda_{A,\gamma,k}$ for $k \in [0, 3]$. Observe that

$$\underbrace{L_{\gamma,0} \oplus \lambda_{A,\gamma,k} \cdot \Delta_A \oplus K[\lambda_{B,\gamma,k}]}_{\text{computed by } \mathcal{P}_A} \oplus M[\lambda_{B,\gamma,k}] = L_{\gamma,0} \oplus \hat{z}_{\gamma,k} \cdot \Delta_A.$$

Hence, in each row of the garbled table for each AND gate $(\alpha, \beta, \gamma, \wedge)$, $\mathcal{P}_A$ additionally encrypts $L_{\gamma,0} \oplus \lambda_{A,\gamma,k} \cdot \Delta_A \oplus K[\lambda_{B,\gamma,k}]$ for $\mathcal{P}_B$ to subsequently compute $L_{\gamma,0} \oplus \hat{z}_{\gamma,k'} \cdot \Delta_A = L_{\gamma,\hat{z}_\gamma}$ where $k' = \hat{z}_\alpha + 2 \cdot \hat{z}_\beta$. $\mathcal{P}_A$ also encrypts $M[\lambda_{A,\gamma,k}]$ and $(M'_j[\lambda_{A,\gamma,k}])_{j=1}^\kappa$ as they are needed for authenticity and public verifiability purposes. Regarding $\mathcal{P}_B$, $\mathcal{P}_B$ also has the corresponding shared randomness $\lambda_{B,\gamma,k}$ of $\lambda_{\gamma,k}$ and their corresponding VOLEitH tags for each row $k \in [0, 3]$. However, only the chosen shared randomness and its corresponding VOLEitH tags are disclosed after evaluating while the garbled circuit; otherwise, $\mathcal{P}_B$'s privacy is compromised. Hence, we enforce $\mathcal{P}_B$ to commit to all $(\lambda_{B,\gamma,k}, (M'_j[\lambda_{B,\gamma,k}])_{j \in [\kappa]})$ by a secure commitment scheme satisfying hiding and binding. Hiding protects $\mathcal{P}_B$'s privacy while binding enforces $\mathcal{P}_B$ to not misbehave.

Discussion for description of $\Pi_{\text{pa-2PC-pre}}$. With the above clarifications for computing garbled tables, as an overview, $\Pi_{\text{pa-2PC-pre}}$ runs as follows.

Initially, similarly to WRK protocol [34], $\mathcal{P}_A$ and $\mathcal{P}_B$ have access to $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ (recalled in Figure 4) to obtain the random values as well as the VOLE correlations. Specifically, as in steps 1, 2, and 3, for $p \in \{A, B\}$, $\mathcal{P}_p$ accesses $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ to obtain Δ_p , the randomness $(\lambda_{p,w})_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}$, as well as their corresponding VOLE correlations w.r.t. $\Delta_{\bar{p}}$. Here, $\mathcal{P}_A$ prepares the labels at all $w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W$ to prepare for garbling. Both parties also obtain the random shared AND triples $(\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{j \in [\kappa], \ell \in [L]}$ where $W = |\mathcal{I}_W|$. Both parties access to $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ to obtain $(\lambda'_{A,\gamma})_{(\alpha,\beta,\gamma,\cdot)}$ and $(\lambda'_{B,\gamma})_{(\alpha,\beta,\gamma,\cdot)}$, respectively, and their corresponding VOLE correlations w.r.t. Δ_B and Δ_A , respectively, such that $\lambda_{A,\gamma} \oplus \lambda'_{B,\gamma} = (\lambda_{A,\alpha} \oplus \lambda_{B,\alpha}) \cdot (\lambda_{A,\beta} \oplus \lambda_{B,\beta})$. Then, the parties employ $\mathcal{F}_{\text{vole-2PC}}$ (c.f. Section 4) to generate VOLEitH tags for those secrets mentioned above as in step 5. Additionally, in step 6, $\mathcal{P}_A$ and $\mathcal{P}_B$ respectively compute VOLEitH tags for those values computed in step 4.

In step 7, $\mathcal{P}_A$ then follows WRK protocol [34] to encrypt the authenticated garbled table. However, in this case, $\mathcal{P}_A$ must additionally commit to $(M'_j[\lambda_{A,\gamma,k}])_{j \in [\kappa]}$ as shown in step 7. The purpose is to enable public auditability via VOLEitH the correct use of $\lambda_{A,\gamma,k}$. On the other hand, to allow public auditability by VOLEitH, for each AND gate, $\mathcal{P}_B$ must commit to every $(\lambda_{B,\gamma,k}, (M'_j[\lambda_{B,\gamma,k}])_{j \in [\kappa]})$ by using randomness $\text{rand}_{\gamma,k}$ for $k \in [0, 3]$. We can do so as follows: (i) In protocol $\Pi_{\text{pa-2PC-eval}}$, $\mathcal{P}_B$ needs to use one among the four values (w.r.t. $k \in [0, 3]$) to evaluate the garbled table, $\mathcal{P}_B$ hence reveals the selected value as well as the VOLEitH tags for auditing purpose. One way is to publish all those mentioned values (without committing to them) and only open the one corresponding to the selected garbled row. However, revealing all those values may help $\mathcal{P}_B$ to recover every row in the garbled table. Therefore, committing to them does help to protect $\mathcal{P}_B$'s secrets. (See Section 2.4 for a preliminary of commitment schemes.) By the hiding property of commitment schemes, $\mathcal{P}_A$ is unable to recover $\mathcal{P}_B$'s secrets; (ii) When evaluating, at each AND gate, $\mathcal{P}_B$ only needs to open (by revealing the corresponding randomness) the commitment to the value that helps evaluate the garbled table while skipping the other three commitments. The binding property of the commitment scheme guarantees that $\mathcal{P}_B$ cannot change the value behind the selected commitment.

Sub-protocol $\Pi_{\text{pa-2PC-eval}}$ (c.f. Figure 10). When both $\mathcal{P}_A$ and $\mathcal{P}_B$ learn their corresponding values w.r.t. wires in $\mathcal{I}_A$ and $\mathcal{I}_B$, respectively, they run protocol $\Pi_{\text{pa-2PC-eval}}$ (c.f. Figure 10) to evaluate and later allow $\mathcal{V}$ to audit the evaluation without leaking secrets of any party except the output of the circuit realizing f .

In **proof preparation**, the parties follow the strategy described in the technical overview (c.f. Section 3.2). In particular, they use the balls-and-bins method for checking AND gates. However, in this step, $\mathcal{P}_A$ and $\mathcal{P}_B$ do not yet have $(\nabla_{p,j})_{p \in \{A,B\}, j \in [\kappa]}$ and the VOLEitH keys. Therefore, they only manipulate the values and the VOLEitH tags. In **input processing**, each $\mathcal{P}_p$ knows the actual values $(z_w)_{w \in \mathcal{I}_p}$. Both parties follow WRK protocol to send and check the authenticated IT-MAC tags w.r.t. Δ_A and Δ_B . Moreover, they also need to send the VOLEitH tags to allow $\mathcal{V}$ to audit. In **circuit evaluation**, $\mathcal{P}_B$ decrypts the rows in the garbled tables. Notice that, at each AND gate, since previously $\mathcal{P}_A$ also encrypts the VOLEitH tags at each garbled table

Proof preparation:

1. $\mathcal{V}$ uniformly samples and publishes permutations $\pi_j : [L] \rightarrow [L] \forall j \in [\kappa]$.
2. $\forall p \in \{A, B\}, \forall j \in [\kappa], \mathcal{P}_p$ permutes as follows.
$$(a_{p,j}^{(\ell)}, b_{p,j}^{(\ell)}, c_{p,j}^{(\ell)}) := (\tilde{a}_{p,j}^{(\pi_j(\ell))}, \tilde{b}_{p,j}^{(\pi_j(\ell))}, \tilde{c}_{p,j}^{(\pi_j(\ell))}),$$

$$(M'_j[a_{p,j}^{(\ell)}], M'_j[b_{p,j}^{(\ell)}], M'_j[c_{p,j}^{(\ell)}]) := (M'_j[\tilde{a}_{p,j}^{(\pi_j(\ell))}], M'_j[\tilde{b}_{p,j}^{(\pi_j(\ell))}], M'_j[\tilde{c}_{p,j}^{(\pi_j(\ell))}]) \forall \ell \in [L].$$
3. $\forall p \in \{A, B\}, \mathcal{P}_p$ publishes
$$(a_{p,j}^{(i)}, b_{p,j}^{(i)}, c_{p,j}^{(i)}, M'_j[a_{p,j}^{(i)}], M'_j[b_{p,j}^{(i)}], M'_j[c_{p,j}^{(i)}])_{j \in [\kappa], i \in [W \cdot \text{bs} + 1, W \cdot \text{bs} + \text{rm}]}.$$
 (8)
4. For $k \in [\text{bs}]$, run step 1 of $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$ (c.f. Figure 5) with inputs
$$((\lambda_{p,\alpha}, \lambda_{p,\beta}, \lambda_{p,\gamma}, M'_j[\lambda_{p,\alpha}], M'_j[\lambda_{p,\beta}], M'_j[\lambda_{p,\gamma}])_{j \in [\kappa]}, \text{all } (\alpha, \beta, \gamma, \wedge),$$

$$(a_{p,j}^{(i)}, b_{p,j}^{(i)}, c_{p,j}^{(i)}, M'_j[a_{p,j}^{(i)}], M'_j[b_{p,j}^{(i)}], M'_j[c_{p,j}^{(i)}])_{j \in [\kappa], i \in [(k-1) \cdot W + 1, k \cdot W]})$$
 from $\mathcal{P}_p$.

Input processing: When $\mathcal{P}_A$ and $\mathcal{P}_B$ have $(z_w)_{w \in \mathcal{I}_A} \in \{0, 1\}^{|\mathcal{I}_A|}$ and $(z_w)_{w \in \mathcal{I}_B} \in \{0, 1\}^{|\mathcal{I}_B|}$, respectively, the parties work as follows. For $p \in \{A, B\}$,

5. $\mathcal{P}_p$ publishes $(\lambda_{p,w}, M[\lambda_{p,w}], (M'_j[\lambda_{p,w}])_{j \in [\kappa]})_{w \in \mathcal{I}_p}$.
 $\mathcal{P}_{\bar{p}}$ checks whether $M[\lambda_{p,w}] = K[\lambda_{p,w}] \oplus \lambda_{p,w} \cdot \Delta_{\bar{p}} \forall w \in \mathcal{I}_{\bar{p}}$.
 $\mathcal{P}_{\bar{p}}$ then publishes $\tilde{z}_w = z_w \oplus \lambda_w = z_w \oplus \lambda_{p,w} \oplus \lambda_{\bar{p},w} \forall w \in \mathcal{I}_{\bar{p}}$.
Finally, $\mathcal{P}_A$ publishes $\mathbf{L}_{w, \tilde{z}_w} \forall w \in \mathcal{I}_A \cup \mathcal{I}_B$.

Circuit evaluation:

6. $\mathcal{P}_B$ evaluates the circuit following a topological order. For each gate $(\alpha, \beta, \gamma, \text{op})$ where $\text{op} \in \{\wedge, \oplus\}$, $\mathcal{P}_B$ knows $(\hat{z}_\alpha, \mathbf{L}_{\alpha, \hat{z}_\alpha})$ and $(\hat{z}_\beta, \mathbf{L}_{\beta, \hat{z}_\beta})$.
If $\text{op} = \oplus$, $\mathcal{P}_B$ computes $\hat{z}_\gamma := \hat{z}_\alpha \oplus \hat{z}_\beta$ and $\mathbf{L}_{\gamma, \hat{z}_\gamma} := \mathbf{L}_{\alpha, \hat{z}_\alpha} \oplus \mathbf{L}_{\beta, \hat{z}_\beta}$.
If $\text{op} = \wedge$, for $k := \hat{z}_\alpha + 2 \cdot \hat{z}_\beta$, $\mathcal{P}_B$ computes and publishes
 $(\lambda_{A,\gamma,k}, M[\lambda_{A,\gamma,k}], (M'_j[\lambda_{A,\gamma,k}])_{j \in [\kappa]}, \mathbf{H}_{\gamma,k}) := G_{\gamma,k} \oplus H(\mathbf{L}_{\alpha, \hat{z}_\alpha}, \mathbf{L}_{\beta, \hat{z}_\beta}, \gamma, k)$. Then, $\mathcal{P}_B$ checks whether
 $M[\lambda_{A,\gamma,k}] = K[\lambda_{A,\gamma,k}] \oplus \lambda_{A,\gamma,k} \cdot \Delta_B$. If so, $\mathcal{P}_B$ computes and publishes $\tilde{z}_\gamma = \lambda_{B,\gamma,k} \oplus \lambda_{A,\gamma,k}$ and
 $\mathbf{L}_{\gamma, \tilde{z}_\gamma} := \mathbf{H}_{\gamma,k} \oplus M[\lambda_{B,\gamma,k}]$.
Eventually, $\mathcal{P}_B$ publishes $(\lambda_{B,\gamma,k}, (M'_j[\lambda_{B,\gamma,k}])_{j \in [\kappa]})$ and $\text{rand}_{\gamma,k}$.

Output determination:

7. $\mathcal{P}_A$ publishes $(\lambda_{A,w}, M[\lambda_{A,w}], (M'_j[\lambda_{A,w}])_{j \in [\kappa]})_{w \in \mathcal{I}_O}$. $\mathcal{P}_B$ checks whether $M[\lambda_{A,w}] = K[\lambda_{A,w}] \oplus \lambda_{A,w} \cdot \Delta_B$. Then, $\mathcal{P}_B$ computes $z_w := \tilde{z}_w \oplus \lambda_{A,w} \oplus \lambda_{B,w}$ and publishes $(\lambda_{B,w}, (M'_j[\lambda_{B,w}])_{j \in [\kappa]})_{w \in \mathcal{I}_O}$ and $(z_w)_{w \in \mathcal{I}_O}$.

Proof proceeding:

8. Each party sends (get) to $\mathcal{F}_{\text{SVOLE-2PC}}$ (c.f. Figure 6) to obtain, $\forall p \in \{A, B\}$, $(\nabla_{p,j})_{j \in [\kappa]}, (K'_j[\lambda_{p,w}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (K'_j[\lambda'_{p,w}])_{w \in \mathcal{I}_W}, (K'_j[\tilde{a}_{p,j}^{(\ell)}], K'_j[\tilde{b}_{p,j}^{(\ell)}], K'_j[\tilde{c}_{p,j}^{(\ell)}])_{j \in [\kappa], \ell \in [L]}$.

Fig. 10. Sub-protocol $\Pi_{\text{pa-2PC-eval}}$. Those highlighted are made public for auditability.

corresponding to each AND gate, the decryption from $\mathcal{P}_B$ hence achieves such VOLEitH tags (say, $M'_j[\lambda_{A,\gamma,k}]$ for $k = \hat{z}_\alpha + 2 \cdot \hat{z}_\beta$). On the other hand, in step 7, $\mathcal{P}_B$ commits to $(\lambda_{B,\gamma,k}, (M'_j[\lambda_{B,\gamma,k}])_{j \in [\kappa]})$ for $k = \hat{z}_\alpha + 2 \cdot \hat{z}_\beta$. Hence, $\mathcal{P}_B$ simply opens by revealing the corresponding randomness. In **output determination**, both parties determine the output values. In **proof proceeding**, $\mathcal{V}$ obtains $(\nabla_{A,j}, \nabla_{B,j})_{j=1}^\kappa$, and the VOLEitH keys from $\mathcal{F}_{\text{SVOLE-2PC}}$ and proceeds to check the published VOLEitH tags obtained above.

Sub-protocol $\Pi_{\text{pa-2PC-ver}}$ (c.f. Figure 11). In this protocol, $\mathcal{V}$ verifies the transcript containing all published messages from $\mathcal{P}_A$ and $\mathcal{P}_B$.

- In steps 1 and 2, when having the VOLEitH keys for $(\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{j \in [\kappa], p \in \{A, B\}}$, permute them and check rm revealed values from $\mathcal{P}_A$ and $\mathcal{P}_B$ (c.f. (8) in Figure 10) satisfying VOLEitH correlations.
- In step 3, compute the VOLEitH keys for intermediate values following the topological order of the circuit.
- In step 4, run the verification of $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$ to verify AND triples following balls-and-bins method [3,35] (discussed in Section 3.2).
- In steps 5, 6, and 7, check the authenticity of revealed randomness via VOLEitH correlations, correct openings of commitments, and correct computation of output bits of the circuit.

Theorem 2 (Security of $\Pi_{\text{pa-2PC}}$). *If H is modeled as a random oracle and COM is secure (see Section 2), $\Pi_{\text{pa-2PC}} = (\Pi_{\text{pa-2PC-pre}}, \Pi_{\text{pa-2PC-eval}}, \Pi_{\text{pa-2PC-ver}})$ (c.f. Figures 9, 10, and 11), securely computes f against malicious adversaries with statistical security $2^{-\rho}$ and soundness error $2^{-\kappa \cdot \tau} + \left(\frac{L}{\text{bs}}\right)^{-\kappa} + \text{negl}(\kappa, \tau)$, where $L = W \cdot \text{bs} + \text{rm}$, in the $\mathcal{F}_{\text{pre}}$ - and $\mathcal{F}_{\text{SVOLE-2PC}}$ -hybrid models where $\text{negl}(\kappa, \tau)$ denotes the negligible probability for breaking the security of components in the protocol, e.g., commitments.*

Verification: Verifier $\mathcal{V}$ proceeds as follows.

1. $\mathcal{V}$ permutes $(K'_j[a_{p,j}^{(\ell)}], K'_j[b_{p,j}^{(\ell)}], K'_j[c_{p,j}^{(\ell)}]) := (K'_j[\tilde{a}_{p,j}^{(\pi_j(\ell))}], K'_j[\tilde{b}_{p,j}^{(\pi_j(\ell))}], K'_j[\tilde{c}_{p,j}^{(\pi_j(\ell))}]) \forall j \in [\kappa], \forall p \in \{A, B\}, \forall \ell \in [L]$.
2. $\forall j, \forall i \in [W \cdot \text{bs} + 1, W \cdot \text{bs} + \text{rm}], \forall p \in \{A, B\}, \mathcal{V}$ checks whether $(a_{A,j}^{(i)} \oplus a_{B,j}^{(i)}) \cdot (b_{A,j}^{(i)} \oplus b_{B,j}^{(i)}) = c_{A,j}^{(i)} \oplus c_{B,j}^{(i)}$,
 $M'_j[a_{p,j}^{(i)}] = K'_j[a_{p,j}^{(i)}] \oplus a_{p,j}^{(i)} \cdot \nabla_{\bar{p},j}, M'_j[b_{p,j}^{(i)}] = K'_j[b_{p,j}^{(i)}] \oplus b_{p,j}^{(i)} \cdot \nabla_{\bar{p},j}$ and $M'_j[c_{p,j}^{(i)}] = K'_j[c_{p,j}^{(i)}] \oplus c_{p,j}^{(i)} \cdot \nabla_{\bar{p},j}$.
3. Following the topological order of the circuit, $\forall p \in \{A, B\}$,
 (a) $\forall (\alpha, \beta, \gamma, \oplus), \forall j \in [\kappa], \mathcal{V}$ computes $K'_j[\lambda_{p,\gamma}] := K'_j[\lambda_{p,\alpha}] \oplus K'_j[\lambda_{p,\beta}]$.
 (b) $\forall (\alpha, \beta, \gamma, \wedge)$, let k be parsed as $k := \hat{z}_\alpha + 2 \cdot \hat{z}_\beta$. $\forall j \in [\kappa], \mathcal{V}$ computes
 $K'_j[\lambda_{p,\gamma,k}] := K'_j[\lambda'_{p,\gamma}] \oplus K'_j[\lambda_{p,\gamma}] \oplus \hat{z}_\alpha \cdot K'_j[\lambda_{p,\beta}] \oplus \hat{z}_\beta \cdot K'_j[\lambda_{p,\alpha}] \oplus u_{\bar{p}} \cdot \hat{z}_\alpha \cdot \hat{z}_\beta \cdot \nabla_{\bar{p},j}$.
4. For $k \in [\text{bs}]$, run step 2 of $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$ (c.f. Figure 5) with inputs
 $((K'_j[\lambda_{p,\alpha}], K'_j[\lambda_{p,\beta}], K'_j[\lambda'_{p,\gamma}])_{j \in [\kappa]}, \text{all } (\alpha, \beta, \gamma, \cdot)),$
 $(K'_j[a_{p,j}^{(i)}], K'_j[b_{p,j}^{(i)}], K'_j[c_{p,j}^{(i)}])_{j \in [\kappa], i \in [(k-1) \cdot W + 1, k \cdot W]}_{p \in \{A, B\}}$
- for verifying the validity of authenticated values used for proving AND gates.
5. $\mathcal{V}$ checks whether all of the followings hold for all $j \in [\kappa]$.
 - $M'_j[\lambda_{p,w}] = K'_j[\lambda_{p,w}] \oplus \lambda_{p,w} \cdot \nabla_{\bar{p},j} \forall w \in \mathcal{I}_{\bar{p}} \cup \mathcal{I}_O \forall p \in \{A, B\}$.
 - $\forall (\alpha, \beta, \gamma, \wedge), M'_j[\lambda_{p,\gamma,k}] = K'_j[\lambda_{p,\gamma,k}] \oplus \lambda_{p,\gamma,k} \cdot \nabla_{\bar{p},j}$ where $k = \hat{z}_\alpha + 2 \cdot \hat{z}_\beta$.
6. $\forall (\alpha, \beta, \gamma, \wedge), \mathcal{V}$ checks whether $\text{cm}_{B,\gamma,k}$ opens to $(\lambda_{B,\gamma,k}, (M'_j[\lambda_{B,\gamma,k}])_j)$ w.r.t. $\text{rand}_{\gamma,k}$ where $k = \hat{z}_\alpha + 2 \cdot \hat{z}_\beta$.
7. As $(\hat{z}_w)_{w \in \mathcal{I}_O}, (z_w)_{w \in \mathcal{I}_O}$ are public, $\mathcal{V}$ checks $z_w = \hat{z}_w \oplus \lambda_{A,w} \oplus \lambda_{B,w} \forall w \in \mathcal{I}_O$.

Fig. 11. Sub-protocol $\Pi_{\text{pa-2PC-ver}}$ for verifying transcript from $(\Pi_{\text{pa-2PC-pre}}, \Pi_{\text{pa-2PC-eval}})$.

The proof of Theorem 2 is deferred to Appendix C. Intuitively, statistical security follows that of the original WRK protocol. Soundness and zero knowledge properties hold according to the employment of VOLEith correlations [6].

Removing $\mathcal{V}$. Notice that, although Δ_A and Δ_B are not revealed, as explained in Section 3.2, $\mathcal{V}$ can verify authenticity of all shared masks via $(\nabla_{A,j}, \nabla_{B,j})_{j=1}^\kappa$ by following the paradigm from [6]. Since $(\nabla_{A,j}, \nabla_{B,j})_{j=1}^\kappa$ are generated via $\mathcal{F}_{\text{sVOLE-2PC}}$ and $\Pi_{\text{sVOLE-2PC}}$ securely realizes $\mathcal{F}_{\text{sVOLE-2PC}}$, we can remove $\mathcal{V}$ by generating the global values $(\nabla_{A,j}, \nabla_{B,j})_{j=1}^\kappa$ via some random oracle (Fiat-Shamir transform [23]) since $\Pi_{\text{sVOLE-2PC}}$ is public-coin. Moreover, we also can generate $(\pi_j)_{j=1}^\kappa$ via random oracles. Hence, we can transform the entire transcript of $\Pi_{\text{pa-2PC}}$ into a non-interactive ZKP which is the tuple $\Sigma_{\text{FS}} = (\text{CMT}, \text{RSP}_1, \text{RSP}_2)$ in ROM as specified in Figure 12 while CH_1 and CH_2 are generated by the random oracles. Here, $\text{vc}_j^{\mathcal{P}_p, \tau, N}$ and $\text{vcopen}_j^{\mathcal{P}_p, \tau, N}$ are a vector commitment and the corresponding all-but-one opening, respectively, employed to generate the VOLEith correlations [32, 6]. Hence, we only require interaction between $\mathcal{P}_A$ and $\mathcal{P}_B$ for generating the NI-ZKP proof. Eventually, we denote by $\Pi_{\text{pa-2PC}}^{\text{FS}}$ the version of $\Pi_{\text{pa-2PC}}$ obtained from replacing $\mathcal{V}$'s challenges by those generated from random oracles, via Fiat-Shamir transform.

Number of rounds of $\Pi_{\text{pa-2PC}}^{\text{FS}}$. Since we apply Fiat-Shamir paradigm, generating VOLEith correlations can be done locally in each party. The interactions involved are those for running authenticated garbling w.r.t. Δ_A and Δ_B following the paradigm from WRK protocol. Notice that the components (VOLEith tags and keys) are usually published together with messages sent from the underlying WRK protocol except that we need an additional round for $\mathcal{P}_B$ to publish $\text{cm}_{B,\gamma,k}$ in step 7 of $\Pi_{\text{pa-2PC-pre}}$. Hence, the total number of rounds of $\Pi_{\text{pa-2PC}}^{\text{FS}}$ is that of WRK protocol plus 1.

Transcript size. We refer to Figures 9, 10, and 12 for notations. The size of CMT is $2 \cdot \kappa \cdot \text{costvc}_{\text{com}}^{\tau, N} + 4 \cdot W \cdot (1 + \rho + \kappa \cdot \tau + |\text{cm}|)$ where $\text{costvc}_{\text{com}}^{\tau, N}$ is introduced in Section 4.2 and $|\text{cm}|$ is the size of $\text{cm}_{B,w,k}$. The size in bits of RSP_1 (where $|\text{rand}|$ is the size of each $\text{rand}_{\gamma,k}$) is

$$\begin{aligned}
 & 2 \cdot \kappa \cdot \text{rm} \cdot (3 + 3 \cdot \tau) + \text{bs} \cdot \underbrace{6 \cdot \kappa \cdot W \cdot (2 \cdot \tau + 1)}_{|\text{msgand}_k|} \\
 & + |\mathcal{I}_A \cup \mathcal{I}_B| \cdot (2 + 2 \cdot \rho + \kappa \cdot \tau) + W \cdot (2 + 3 \cdot \rho + 2 \cdot \kappa \cdot \tau + |\text{rand}|) \\
 & + |\mathcal{I}_O| \cdot (3 + \rho + 2 \cdot \kappa \cdot \tau).
 \end{aligned}$$

The size of RSP_2 is $2 \cdot \kappa \cdot \text{costvc}_{\text{open}}^{\tau, N}$ bits where $\text{costvc}_{\text{open}}^{\tau, N}$ is introduced in Section 4.2. Hence, to simplify, the size of Σ_{FS} is bounded by

$$\mathcal{O} \left(\kappa \cdot \text{costvc}_{\text{total}}^{\tau, N} + |\text{cm}| \cdot W + \kappa \cdot \tau \cdot (W \cdot \text{bs} + \text{rm}) + (\rho + \kappa \cdot \tau) \cdot (|\mathcal{I}_A| + |\mathcal{I}_B| + W + |\mathcal{I}_O|) \right)$$

$$\begin{aligned}
\text{CMT} &= \left((\text{vc}_j^{\mathcal{P}_p, \tau, N})_{p \in \{A, B\}, j \in [\kappa]}, (G_{w, k}, \text{cm}_{B, w, k})_{w \in \mathcal{I}_W, k \in [0, 3]} \right), \\
\text{CH}_1 &= (\pi_j)_{j \in [\kappa]} \leftarrow h_{\text{FS}}^{(1)}(\kappa, \rho, \tau, \text{bs}, \text{rm}, |\mathcal{I}_A|, |\mathcal{I}_B|, W, |\mathcal{I}_O|, \text{CMT}), \\
\text{RSP}_1 &= \left((a_{p, j}^{(i)}, b_{p, j}^{(i)}, c_{p, j}^{(i)}, M'_j[a_{p, j}^{(i)}], M'_j[b_{p, j}^{(i)}], M'_j[c_{p, j}^{(i)}])_{p \in \{A, B\}, j \in [\kappa], i \in [W \cdot \text{bs} + 1, W \cdot \text{bs} + \text{rm}]}, \right. \\
&\quad (\text{msgand}_k)_{k=1}^{\text{bs}}, (\lambda_{p, w}, M[\lambda_{p, w}], (M'_j[\lambda_{p, w}])_{j=1}^{\kappa}, \hat{z}_w, \mathbf{L}_{w, \hat{z}_w})_{p \in \{A, B\}, w \in \mathcal{I}_{\overline{P}}}, \\
&\quad (\lambda_{A, \gamma, k}, M[\lambda_{A, \gamma, k}], (M'_j[\lambda_{A, \gamma, k}])_{j=1}^{\kappa}, \mathbf{H}_{\gamma, k}, \hat{z}_{\gamma}, \mathbf{L}_{\gamma, \hat{z}_{\gamma}}), \\
&\quad \left. \lambda_{B, \gamma, k}, (M'_j[\lambda_{B, \gamma, k}])_{j=1}^{\kappa}, \text{rand}_{\gamma, k} \right)_{\text{all } (\alpha, \beta, \gamma, \wedge) \text{ and } k = \hat{z}_{\alpha} + 2 \cdot \hat{z}_{\beta}}, \\
&\quad (\lambda_{A, w}, M[\lambda_{A, w}], (M'_j[\lambda_{A, w}])_{j=1}^{\kappa}, \lambda_{B, w}, (M'_j[\lambda_{B, w}])_{j=1}^{\kappa}, z_w)_{w \in \mathcal{I}_O}), \\
\text{CH}_2 &= (\nabla_{A, j}, \nabla_{B, j})_{j=1}^{\kappa} \leftarrow h_{\text{FS}}^{(2)}(\text{CH}_1, \text{RSP}_1), \\
\text{RSP}_2 &= (\text{vcopen}_j^{\mathcal{P}_p, \tau, N})_{p \in \{A, B\}, j \in [\kappa]},
\end{aligned}$$

where (i) $\text{vc}_j^{\mathcal{P}, \tau, N}$ and $\text{vcopen}_j^{\mathcal{P}, \tau, N}$ are the vector commitment and its all-but-one opening (recalled in the full version [29]) for generating VOLEitH correlations in the j -th call to $\Pi_{\text{sVOLE-2PC}}$ by $\mathcal{P} \in \{\mathcal{P}_A, \mathcal{P}_B\}$ and $\mathcal{V}$; and (ii) msgand_k is the message published from running step 1 in the k -th call to $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$.

Fig. 12. Transcript of $(\Pi_{\text{pa-2PC-pre}}, \Pi_{\text{pa-2PC-eval}})$ in Figures 9 and 10.

where $\text{costvc}_{\text{total}}^{\tau, N} = \text{costvc}_{\text{com}}^{\tau, N} + \text{costvc}_{\text{open}}^{\tau, N}$ as defined in Section 2.2. The sizes of $\text{costvc}_{\text{total}}^{\tau, N}$ [6] and $|\text{cm}|$ are relatively small if computed from hash functions and PRGs. From Weng et al. [35], a careful setting of rm and bs can achieve L asymptotically bounded by $\mathcal{O}(W)$. Specifically, by setting $\text{rm} = \text{bs}$, $\left(\frac{L}{\text{bs}}\right)^{\kappa} \geq W^{\text{bs} \cdot \kappa}$. If we want to bound $\left(\frac{L}{\text{bs}}\right)^{-\kappa} \leq 2^{-\varphi}$ for some φ , then it must be that $W^{\text{bs} \cdot \kappa} \geq 2^{\varphi}$, or equivalently $\text{bs} \geq \varphi / (\kappa \cdot \log_2 W)$ which is relatively small as ρ , κ , and $\log_2 W$ are small, e.g., not exceeding 30. Hence, with an appropriate setting of parameters, the above efficiency is dominated by $\mathcal{O}((\rho + \kappa \cdot \tau) \cdot (|\mathcal{I}_A| + |\mathcal{I}_B| + W + |\mathcal{I}_O|))$. Choosing $\rho \approx \kappa \cdot \tau$, our proof size is asymptotically about the transcript size of WRK's protocol.

Changelog

We present the changelog as follows.

2025-Apr-30: Upload the initial manuscript to Cryptology ePrint Archive.

2026-Jan-10: Revise the manuscript to improve editorial quality, and correct grammatical errors and typos.

References

1. Aguilar-Melchor, C., Gama, N., Howe, J., Hülsing, A., Joseph, D., Yue, D.: The Return of the SDitH. In: EUROCRYPT 2023 (2023)
2. Aguilar-Melchor, C., Hülsing, A., Joseph, D., Majenz, C., Ronen, E., Yue, D.: SDitH in the QROM. In: ASIACRYPT 2023 (2023)
3. Araki, T., Barak, A., Furukawa, J., Lichter, T., Lindell, Y., Nof, A., Ohara, K., Watzman, A., Weinstein, O.: Optimized Honest-Majority MPC for Malicious Adversaries — Breaking the 1 Billion-Gate Per Second Barrier. In: S&P 2017 (2017)
4. Baum, C., Braun, L., Munch-Hansen, A., Razet, B., Scholl, P.: Appenzeller to Brie: Efficient Zero-Knowledge Proofs for Mixed-Mode Arithmetic and Z2k. In: CCS 2021 (2021)
5. Baum, C., Braun, L., Munch-Hansen, A., Scholl, P.: MozZ_{2k}arella: Efficient Vector-OLE and Zero-Knowledge Proofs over Z_{2k}. In: CRYPTO 2022 (2022)
6. Baum, C., Braun, L., de Saint Guilhem, C.D., Kloof, M., Orsini, E., Roy, L., Scholl, P.: Publicly Verifiable Zero-Knowledge and Post-Quantum Signatures from VOLE-in-the-Head. In: CRYPTO 2023 (2023)
7. Baum, C., Damgård, I., Orlandi, C.: Publicly Auditable Secure Multi-Party Computation. In: SCN 2014 (2014)
8. Baum, C., Malozemoff, A.J., Rosen, M.B., Scholl, P.: Mac'n'Cheese: Zero-Knowledge Proofs for Boolean and Arithmetic Circuits with Nested Disjunctions. In: CRYPTO 2021 (2021)
9. Baum, C., Nof, A.: Concretely-Efficient Zero-Knowledge Arguments for Arithmetic Circuits and Their Application to Lattice-Based Cryptography. In: PKC 2020 (2020)

10. Baum, C., Orsini, E., Scholl, P., Soria-Vazquez, E.: Efficient constant-round MPC with identifiable abort and public verifiability. In: CRYPTO 2020 (2020)
11. Baum, C., de Saint Guilhem, C.D., Kales, D., Orsini, E., Scholl, P., Zaverucha, G.: Banquet: Short and Fast Signatures from AES. In: PKC 2021 (2021)
12. Beaver, D.: Efficient Multiparty Protocols Using Circuit Randomization. In: CRYPTO 1991 (1992)
13. Bui, D.: Shorter VOLEitH Signature from Multivariate Quadratic. ePrint Archive (2024), <https://ia.cr/2024/465>
14. Bui, D., Carozza, E., Couteau, G., Goudarzi, D., Joux, A.: Faster Signatures from MPC-in-the-Head. In: ASIACRYPT 2024 (2025)
15. Carozza, E., Couteau, G., Joux, A.: Short Signatures from Regular Syndrome Decoding in the Head. In: EUROCRYPT 2023 (2023)
16. Chase, M., Derler, D., Goldfeder, S., Orlandi, C., Ramacher, S., Rechberger, C., Slamanig, D., Zaverucha, G.: Post-Quantum Zero-Knowledge and Signatures from Symmetric-Key Primitives. In: CCS 2017 (2017)
17. Cui, H., Liu, H., Yan, D., Yang, K., Yu, Y., Zhang, K.: ReSolveD: Shorter Signatures from Regular Syndrome Decoding and VOLE-in-the-Head. In: PKC 2024 (2024)
18. Cui, H., Wang, X., Yang, K., Yu, Y.: Actively Secure Half-Gates with Minimum Overhead Under Duplex Networks. In: EUROCRYPT 2023 (2023)
19. Dittmer, S., Ishai, Y., Lu, S., Ostrovsky, R.: Improving Line-Point Zero Knowledge: Two Multiplications for the Price of One. In: CCS 2022 (2022)
20. Dittmer, S., Ishai, Y., Ostrovsky, R.: Line-Point Zero Knowledge and Its Applications. In: ITC 2021 (2021)
21. Evans, D., Kolesnikov, V., Rosulek, M.: A Pragmatic Introduction to Secure Multi-Party Computation. Found. Trends Priv. Secur. **2**(2-3), 70–246 (2018). <https://doi.org/10.1561/33000000019>
22. Feneuil, T., Joux, A., Rivain, M.: Syndrome Decoding in the Head: Shorter Signatures from Zero-Knowledge Proofs. In: CRYPTO 2022 (2022)
23. Fiat, A., Shamir, A.: How To Prove Yourself: Practical Solutions to Identification and Signature Problems. In: CRYPTO 1986 (1987)
24. Giacomelli, I., Madsen, J., Orlandi, C.: ZKBoo: Faster Zero-Knowledge for Boolean Circuits. In: USENIX Security 2016 (2016)
25. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-knowledge from secure multiparty computation. In: STOC 2007 (2007)
26. Jain, A., Krenn, S., Pietrzak, K., Tentes, A.: Commitments and Efficient Zero-Knowledge Proofs from Learning Parity with Noise. In: ASIACRYPT 2012 (2012)
27. Katz, J., Kolesnikov, V., Wang, X.: Improved Non-Interactive Zero Knowledge with Applications to Post-Quantum Signatures. In: CCS 2018 (2018)
28. Katz, J., Ranellucci, S., Rosulek, M., Wang, X.: Optimizing Authenticated Garbling for Faster Secure Two-Party Computation. In: CRYPTO 2018 (2018)
29. Ling, S., Ngo, C.N., Tang, K.H., Wang, H.: Publicly Auditable Garbled Circuit. ePrint Archive (2025), <https://ia.cr/2025/772>
30. Ouyang, Y., Tang, D., Xu, Y.: Code-Based Zero-Knowledge from VOLE-in-the-Head and Their Applications: Simpler, Faster, and Smaller. In: ASIACRYPT 2024 (2025)
31. Ozdemir, A., Boneh, D.: Experimenting with Collaborative zk-SNARKs: Zero-Knowledge Proofs for Distributed Secrets. In: USENIX Security 2022 (2022)
32. Roy, L.: SoftSpokenOT: Quieter OT Extension from Small-Field Silent VOLE in the Minicrypt Model. In: CRYPTO 2022 (2022)
33. Delpech de Saint Guilhem, C., Orsini, E., Tanguy, T.: Limbo: Efficient Zero-knowledge MPCitH-based Arguments. In: CCS 2021 (2021)
34. Wang, X., Ranellucci, S., Katz, J.: Authenticated Garbling and Efficient Maliciously Secure Two-Party Computation. In: CCS 2017 (2017)
35. Weng, C., Yang, K., Katz, J., Wang, X.: Wolverine: Fast, Scalable, and Communication-Efficient Zero-Knowledge Proofs for Boolean and Arithmetic Circuits. In: S&P 2021 (2021)
36. Weng, C., Yang, K., Xie, X., Katz, J., Wang, X.: Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning. In: USENIX Security 2021 (2021)
37. Yang, K., Sarkar, P., Weng, C., Wang, X.: QuickSilver: Efficient and Affordable Zero-Knowledge Proofs for Circuits and Polynomials over Any Field. In: CCS 2021 (2021)

A WRK Protocol

We recall WRK protocol $\Pi_{2PC}^{\mathcal{P}_A, \mathcal{P}_B}$ [34] in Figure 13.

Inputs: Both parties $\mathcal{P}_A$ and $\mathcal{P}_B$ agree on a common circuit for function $f : \{0, 1\}^{|\mathcal{I}_A|} \times \{0, 1\}^{|\mathcal{I}_B|} \rightarrow \{0, 1\}^{|\mathcal{I}_O|}$. In the input-preprocessing phase, $\mathcal{P}_A$ and $\mathcal{P}_B$ holds $\mathbf{x} = (z_i)_{i \in \mathcal{I}_A} \in \{0, 1\}^{|\mathcal{I}_A|}$ and $\mathbf{y} = (z_i)_{i \in \mathcal{I}_B} \in \{0, 1\}^{|\mathcal{I}_B|}$, respectively. Moreover, both parties also agree on a common parameter $\rho \in \mathbb{N}$.

Function-independent preprocessing:

- $\forall p \in \{A, B\}$, $\mathcal{P}_p$ sends (init) to $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ to receive $\Delta_p \in \mathbb{F}_{2^\rho}$.
- $\forall w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W$, both parties send random to $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$. In return, $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ sends $(\lambda_{p,w}, M[\lambda_{p,w}], K[\lambda_{p,w}])$ to $\mathcal{P}_p$ for $p \in \{A, B\}$. Define $\lambda_w := \lambda_{A,w} \oplus \lambda_{B,w}$. $\mathcal{P}_A$ samples $L_{w,0} \xleftarrow{\$} \mathbb{F}_{2^\rho}$.

Function-dependent preprocessing:

- $\forall (\alpha, \beta, \gamma, \oplus)$, $\forall p \in \{A, B\}$, $\mathcal{P}_p$ computes $(\lambda_{p,\gamma}, M[\lambda_{p,\gamma}], K[\lambda_{p,\gamma}]) := (\lambda_{p,\alpha} \oplus \lambda_{p,\beta}, M[\lambda_{p,\alpha}] \oplus M[\lambda_{p,\beta}], K[\lambda_{p,\alpha}] \oplus K[\lambda_{p,\beta}])$. Additionally, $\mathcal{P}_A$ computes $L_{\gamma,0} := L_{\alpha,0} \oplus L_{\beta,0}$.
- $\forall (\alpha, \beta, \gamma, \wedge)$:
 - $\forall p \in \{A, B\}$, $\mathcal{P}_p$ sends (and, $(\lambda_{p,\alpha}, M[\lambda_{p,\alpha}], K[\lambda_{p,\alpha}]), (\lambda_{p,\beta}, M[\lambda_{p,\beta}], K[\lambda_{p,\beta}])$) to $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$. Then, $\mathcal{F}_{\text{pre}}^{\mathcal{P}_A, \mathcal{P}_B}$ sends $(\lambda'_{p,\gamma}, M[\lambda'_{p,\gamma}], K[\lambda'_{p,\gamma}])$ to $\mathcal{P}_p$. Here, $\lambda'_{A,\gamma} \oplus \lambda'_{B,\gamma} = \lambda_\alpha \cdot \lambda_\beta$.
 - $\forall k \in [0, 3]$, let $(k_0, k_1) := \text{bin}_2(k) \in \{0, 1\}^2$, $u_A = 1$, and $u_B = 0$. $\forall p \in \{A, B\}$, $\mathcal{P}_p$ computes
$$\lambda_{p,\gamma,k} := \lambda'_{p,\gamma} \oplus \lambda_{p,\gamma} \oplus k_0 \cdot \lambda_{p,\beta} \oplus k_1 \cdot \lambda_{p,\alpha} \oplus u_p \cdot k_0 \cdot k_1,$$

$$M[\lambda_{p,\gamma,k}] := M[\lambda'_{p,\gamma}] \oplus M[\lambda_{p,\gamma}] \oplus k_0 \cdot M[\lambda_{p,\beta}] \oplus k_1 \cdot M[\lambda_{p,\alpha}],$$

$$K[\lambda_{p,\gamma,k}] := K[\lambda'_{p,\gamma}] \oplus K[\lambda_{p,\gamma}] \oplus k_0 \cdot K[\lambda_{p,\beta}] \oplus k_1 \cdot K[\lambda_{p,\alpha}] \oplus u_p \cdot k_0 \cdot k_1 \cdot \Delta_p.$$
- $\mathcal{P}_A$ computes $L_{\alpha,1} := L_{\alpha,0} \oplus \Delta_A$ and $L_{\beta,1} := L_{\beta,0} \oplus \Delta_A$. $\mathcal{P}_A$ sends the following to $\mathcal{P}_B$, $\forall k \in [0, 3]$ with $(k_0, k_1) := \text{bin}_2(k)$:
$$G_{\gamma,k} := H(L_{\alpha,k_0}, L_{\beta,k_1}, \gamma, k) \oplus (\lambda_{A,\gamma,k}, M[\lambda_{A,\gamma,k}], L_{\gamma,0} \oplus K[\lambda_{B,\gamma,k}] \oplus \lambda_{A,\gamma,k} \cdot \Delta_A).$$

Input processing: For $p \in \{A, B\}$,

- For $w \in \mathcal{I}_p$, $\mathcal{P}_p$ sends $(\lambda_{p,w}, M[\lambda_{p,w}])$ to $\mathcal{P}_p$. $\mathcal{P}_p$ checks whether $M[\lambda_{p,w}] = K[\lambda_{p,w}] \oplus \lambda_{p,w} \cdot \Delta_p$. $\mathcal{P}_p$ then sends $\hat{z}_w = z_w \oplus \lambda_w = z_w \oplus \lambda_{p,w} \oplus \lambda_{p,w}$ to $\mathcal{P}_p$.
- Finally, $\mathcal{P}_A$ sends $L_{w,\hat{z}_w} \forall w \in \mathcal{I}_A \cup \mathcal{I}_B$ to $\mathcal{P}_B$.

Circuit evaluation:

- $\mathcal{P}_B$ evaluates the circuit following a topological order. $\forall (\alpha, \beta, \gamma, \text{op})$ where $\text{op} \in \{\wedge, \oplus\}$, $\mathcal{P}_B$ holds $(\hat{z}_\alpha, L_{\alpha,\hat{z}_\alpha})$ and $(\hat{z}_\beta, L_{\beta,\hat{z}_\beta})$.
 - If $\text{op} = \oplus$, $\mathcal{P}_B$ computes $\hat{z}_\gamma := \hat{z}_\alpha \oplus \hat{z}_\beta$ and $L_{\gamma,\hat{z}_\gamma} := L_{\alpha,\hat{z}_\alpha} \oplus L_{\beta,\hat{z}_\beta}$.
 - If $\text{op} = \wedge$, $\mathcal{P}_B$ computes $i := \hat{z}_\alpha + 2 \cdot \hat{z}_\beta$. Then, $\mathcal{P}_B$ computes $(\lambda_{A,\gamma,i}, M[\lambda_{A,\gamma,i}], L_{\gamma,0} \oplus K[\lambda_{B,\gamma,i}] \oplus \lambda_{A,\gamma,i} \cdot \Delta_A) := G_{\gamma,i} \oplus H(L_{\alpha,\hat{z}_\alpha}, L_{\beta,\hat{z}_\beta}, \gamma, i)$. $\mathcal{P}_B$ checks whether $M[\lambda_{A,\gamma,i}] = K[\lambda_{A,\gamma,i}] \oplus \lambda_{A,\gamma,i} \cdot \Delta_B$. If so, $\mathcal{P}_B$ computes $\hat{z}_\gamma = \lambda_{B,\gamma,i} \oplus \lambda_{A,\gamma,i}$ and $L_{\gamma,\hat{z}_\gamma} := (L_{\gamma,0} \oplus K[\lambda_{B,\gamma,i}] \oplus \lambda_{A,\gamma,i} \cdot \Delta_A) \oplus M[\lambda_{B,\gamma,i}]$.

Output determination:

- For $w \in \mathcal{I}_O$, $\mathcal{P}_A$ sends $(\lambda_{A,w}, M[\lambda_{A,w}])$ to $\mathcal{P}_B$ who checks whether $M[\lambda_{A,w}] = L[\lambda_{A,w}] \oplus \lambda_{A,w} \cdot \Delta_B$. If so, $\mathcal{P}_B$ computes $z_w := \hat{z}_w \oplus \lambda_{A,w} \oplus \lambda_{B,w}$.

Fig. 13. Protocol $\Pi_{2\text{PC}}^{\mathcal{P}_A, \mathcal{P}_B}$ [34, Figure 2].

B Proof Sketch of Theorem 1

Proof (Proof Sketch of Theorem 1). The security is guaranteed by the following cases.

Malicious $\tilde{\mathcal{P}}_B$. We construct a simulator $\mathcal{S}_A$ that has access to ideal functionality $\mathcal{F}_{\text{sVOLE-2PC}}$, plays the role of the honest $\mathcal{P}_A$ and $\mathcal{V}$ and runs the code of $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}_A, \mathcal{V}, \tau, N}$ and $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}_B, \mathcal{V}, \tau, N}$, as a series of hybrid experiments, as follows.

1. **Hybrid 1.** $\mathcal{S}_A$ runs the code as specified in $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}_A, \mathcal{V}, \tau, N}$ and $\mathcal{F}_{\text{sVOLE}}^{\mathcal{P}_B, \mathcal{V}, \tau, N}$.
2. **Hybrid 2.** This is the as Hybrid 1, except that $\hat{r}_{w,j}$, $\hat{r}'_{w,j}$, $\hat{a}_{A,j}^{(\ell)}$, $\hat{b}_{A,j}^{(\ell)}$, $\hat{c}_{A,j}^{(\ell)}$ are sampled uniformly. Then, compute $\bar{r}_{w,j} := \hat{r}_{w,j} \oplus \lambda_{A,w}$, $\bar{r}'_{w,j} := \hat{r}'_{w,j} \oplus \lambda'_{A,w}$, $\bar{a}_{A,j}^{(\ell)} := \hat{a}_{A,j}^{(\ell)} \oplus \hat{a}_{A,j}^{(\ell)}$, $\bar{b}_{A,j}^{(\ell)} := \hat{b}_{A,j}^{(\ell)} \oplus \hat{b}_{A,j}^{(\ell)}$, $\bar{c}_{A,j}^{(\ell)} := \hat{c}_{A,j}^{(\ell)} \oplus \hat{c}_{A,j}^{(\ell)}$. Their respective VOLEitH tags are computed accordingly based on the homomorphism of VOLEitH correlations. This has the same distribution as Hybrid 1.
3. **Hybrid 3.** This is the same as Hybrid 2 except that $\mathcal{S}_A$ sets $\lambda_{A,w} := 0$, $\lambda'_{A,w} := 0$, $\tilde{a}_{A,j}^{(\ell)} := 0$, $\tilde{b}_{A,j}^{(\ell)} := 0$, $\tilde{c}_{A,j}^{(\ell)} := 0$. Here, although the inputs from $\mathcal{P}_A$ are all set to be 0, this hybrid is still indistinguishable from Hybrid 2 because $\bar{r}_{w,j} \oplus \lambda_{A,w}$, $\bar{r}'_{w,j} \oplus \lambda'_{A,w}$, $\bar{a}_{A,j}^{(\ell)} \oplus \tilde{a}_{A,j}^{(\ell)}$, $\bar{b}_{A,j}^{(\ell)} \oplus \tilde{b}_{A,j}^{(\ell)}$, $\bar{c}_{A,j}^{(\ell)} \oplus \tilde{c}_{A,j}^{(\ell)}$ have the same distribution with those in Hybrid 2.

Hence, we see that the real execution between $\tilde{\mathcal{P}}_{\mathcal{B}}$ and other parties is indistinguishable from the simulated execution between $\tilde{\mathcal{P}}_{\mathcal{B}}$ and $\mathcal{S}_{\mathcal{A}}$ since, in Hybrid 3, even $\mathcal{S}_{\mathcal{A}}$ sets all inputs of $\mathcal{P}_{\mathcal{A}}$ to be 0.

Malicious $\tilde{\mathcal{P}}_{\mathcal{A}}$. We construct a simulator $\mathcal{S}_{\mathcal{B}}$ that has access to ideal functionality $\mathcal{F}_{\text{SVOLE-2PC}}$, plays the role of the honest $\mathcal{P}_{\mathcal{B}}$ and $\mathcal{V}$, and runs the code of $\mathcal{F}_{\text{SVOLE}}^{\mathcal{P}_{\mathcal{A}}, \mathcal{V}, \tau, N}$ similarly to $\mathcal{S}_{\mathcal{A}}$.

Thus, we conclude the proof. $\square$

C Proof of Theorem 2

Proof (Proof of Theorem 2). Correctness is straightforward. We consider the following cases.

For the cases of malicious $\tilde{\mathcal{P}}_{\mathcal{A}}$ and $\tilde{\mathcal{P}}_{\mathcal{B}}$, the proofs follow the paradigm from [34]. Here, we focus on constructing a simulator $\mathcal{S}_{\text{zk}}$ for simulating the transcript from the view of a semi-honest verifier and an extractor $\mathcal{E}$ for extracting the witnesses of $\mathcal{P}_{\mathcal{A}}$ and $\mathcal{P}_{\mathcal{B}}$, hence implying the knowledge soundness of our combined protocol $\Pi_{\text{pa-2PC}} = (\Pi_{\text{pa-2PC-pre}}, \Pi_{\text{pa-2PC-eval}}, \Pi_{\text{pa-2PC-ver}})$ (c.f. Figures 9, 10, and 11).

Semi-honest $\mathcal{V}$. We construct the simulator $\mathcal{S}_{\text{zk}}$ as follows. $\mathcal{S}_{\text{zk}}$ receives the inputs as specified for the case that $\mathcal{V}$ is corrupted in Figure 6. In particular, it receives $(\nabla_{\mathcal{A},j}, \nabla_{\mathcal{B},j})_{j \in [\kappa]}$, and the sequences of VOLEitH keys $(K'_j[\lambda_{p,w}])_{w \in \mathcal{I}_{\mathcal{A}} \cup \mathcal{I}_{\mathcal{B}} \cup \mathcal{I}_{\mathcal{W}}}$, $(K'_j[\lambda'_{p,w}])_{w \in \mathcal{I}_{\mathcal{W}}}$, and $(K'_j[\tilde{a}_{p,j}^{(\ell)}], K'_j[\tilde{b}_{p,j}^{(\ell)}], K'_j[\tilde{c}_{p,j}^{(\ell)}])_{\ell \in [L], j \in [\kappa]}$ for $p \in \{\mathcal{A}, \mathcal{B}\}$.

Additionally, $\mathcal{S}_{\text{zk}}$ also receives permutations $(\pi_j)_{j \in [\kappa]}$ for proving satisfaction of AND gates. We now separate the strategy for $\mathcal{S}_{\text{zk}}$ into the following two cases, namely, of garbling and of AND gates.

The case of garbling. Notice that $\mathcal{V}$ cannot possess $\Delta_{\mathcal{A}}$ and $\Delta_{\mathcal{B}}$ which are secrets of $\mathcal{P}_{\mathcal{A}}$ and $\mathcal{P}_{\mathcal{B}}$, respectively. Therefore, $\mathcal{V}$ cannot proceed with the checks of valid VOLE tuples, w.r.t. $\Delta_{\mathcal{A}}$ and $\Delta_{\mathcal{B}}$, when decrypting the garbled tables. Moreover, we do not care whether $(\lambda_{\mathcal{A},\alpha} \oplus \lambda_{\mathcal{B},\alpha}) \cdot (\lambda_{\mathcal{A},\beta} \oplus \lambda_{\mathcal{B},\beta}) = \lambda'_{\mathcal{A},\gamma} \oplus \lambda'_{\mathcal{B},\gamma}$ for any AND gate $(\alpha, \beta, \gamma, \wedge)$ as $\mathcal{S}_{\text{zk}}$ can simulate the checks for the AND gates, to be discussed below. On the other hand, all selected and decrypted rows of the garbled tables and the labels determined by $\mathcal{P}_{\mathcal{A}}$ during evaluation can be published without compromising the privacy of both $\mathcal{P}_{\mathcal{A}}$ and $\mathcal{P}_{\mathcal{B}}$.

By the above observations, we construct $\mathcal{S}_{\text{zk}}$ as follows.

1. Sample $\Delta_{\mathcal{A}}$ and $\Delta_{\mathcal{B}}$ uniformly.
2. Find a random sequence $(\mathfrak{z}_w)_{w \in \mathcal{I}_{\mathcal{A}} \cup \mathcal{I}_{\mathcal{B}} \cup \mathcal{I}_{\mathcal{W}}}$ such that, (i) for any XOR gate $(\alpha, \beta, \gamma, \oplus)$, $\mathfrak{z}_{\gamma} = \mathfrak{z}_{\alpha} \oplus \mathfrak{z}_{\beta}$, and, (ii) for any $w \in \mathcal{I}_{\mathcal{O}}$, $\mathfrak{z}_w = z_w$. This sequence $(\mathfrak{z}_w)_{w \in \mathcal{I}_{\mathcal{A}} \cup \mathcal{I}_{\mathcal{B}} \cup \mathcal{I}_{\mathcal{W}}}$ contains the simulated values that $\mathcal{S}_{\text{zk}}$ generates as $\mathcal{S}_{\text{zk}}$ does not know the actual witnesses of both $\mathcal{P}_{\mathcal{A}}$ and $\mathcal{P}_{\mathcal{B}}$. Here, we do not care whether $\mathfrak{z}_{\gamma} = \mathfrak{z}_{\alpha} \cdot \mathfrak{z}_{\beta}$ for any AND gate $(\alpha, \beta, \gamma, \wedge)$.
3. For each $w \in \mathcal{I}_{\mathcal{A}} \cup \mathcal{I}_{\mathcal{B}} \cup \mathcal{I}_{\mathcal{W}}$, $\mathcal{S}_{\text{zk}}$ works as follows.
 - (a) Sample $\lambda_{\mathcal{A},w}, \lambda_{\mathcal{B},w} \xleftarrow{\$} \{0, 1\}$. Let $\lambda_w = \lambda_{\mathcal{A},w} \oplus \lambda_{\mathcal{B},w}$ and $\hat{\lambda}_w = \mathfrak{z}_w \oplus \lambda_w = \mathfrak{z}_w \oplus \lambda_{\mathcal{A},w} \oplus \lambda_{\mathcal{B},w}$ be the simulated masked value at wire w .
 - (b) Sample $M[\lambda_{p,w}]$ and $K[\lambda_{p,w}]$, for $p \in \{\mathcal{A}, \mathcal{B}\}$, such that $M[\lambda_{p,w}] = K[\lambda_{p,w}] \oplus \lambda_{p,w} \cdot \Delta_{\bar{p}}$.
 - (c) For $j \in [\kappa]$, compute $M'_j[\lambda_{p,w}]$, for $p \in \{\mathcal{A}, \mathcal{B}\}$, such that $M'_j[\lambda_{p,w}] = K'_j[\lambda_{p,w}] \oplus \lambda_{p,w} \cdot \nabla_{\bar{p},j}$.
4. For each AND gate $(\alpha, \beta, \gamma, \wedge)$, notice that, in the real protocol $\Pi_{\text{pa-2PC-eval}}$, $\mathcal{P}_{\mathcal{B}}$ can determine the row $k = \hat{z}_{\alpha} + 2 \cdot \hat{z}_{\beta}$ to decrypt the respective row of the garbled table from the real masked values $\hat{z}_{\alpha}$ and $\hat{z}_{\beta}$. Hence, $\mathcal{S}_{\text{zk}}$ can similarly determine $k = \hat{\mathfrak{z}}_{\alpha} + 2 \cdot \hat{\mathfrak{z}}_{\beta}$ from the simulated masked values $\hat{\mathfrak{z}}_{\alpha}$ and $\hat{\mathfrak{z}}_{\beta}$. Therefore, when simulating the garbled tables, $\mathcal{S}_{\text{zk}}$ must guarantee that, from the decryption, one can obtain $\hat{\mathfrak{z}}_{\gamma} = \hat{z}_{\gamma} \oplus \lambda_{\mathcal{A},\gamma} \oplus \lambda_{\mathcal{B},\gamma}$. This is the crucial point for our sampling of $\lambda'_{\mathcal{B},\gamma}$ and $\lambda'_{\mathcal{A},\gamma}$. As said above, we do not care whether $(\lambda_{\mathcal{A},\alpha} \oplus \lambda_{\mathcal{B},\alpha}) \cdot (\lambda_{\mathcal{A},\beta} \oplus \lambda_{\mathcal{B},\beta}) = \lambda'_{\mathcal{A},\gamma} \oplus \lambda'_{\mathcal{B},\gamma}$. Hence, $\mathcal{S}_{\text{zk}}$ samples $\lambda'_{\mathcal{A},\gamma}$ and $\lambda'_{\mathcal{B},\gamma}$ such that

$$(\lambda'_{\mathcal{A},\gamma} \oplus \lambda'_{\mathcal{B},\gamma}) \oplus \lambda_{\gamma} \oplus \hat{\mathfrak{z}}_{\alpha} \cdot \lambda_{\beta} \oplus \hat{\mathfrak{z}}_{\beta} \cdot \lambda_{\alpha} = \hat{\mathfrak{z}}_{\gamma}$$

where $\lambda_{\alpha} = \lambda_{\mathcal{A},\alpha} \oplus \lambda_{\mathcal{B},\alpha}$ and $\lambda_{\beta} = \lambda_{\mathcal{A},\beta} \oplus \lambda_{\mathcal{B},\beta}$. With $\lambda'_{\mathcal{A},\gamma}$ and $\lambda'_{\mathcal{B},\gamma}$ in hand, $\mathcal{S}_{\text{zk}}$ computes $M[\lambda'_{p,\gamma}]$ and $M'_j[\lambda'_{p,\gamma}]$, for $p \in \{\mathcal{A}, \mathcal{B}\}$, such that

$$\begin{cases} M[\lambda'_{p,\gamma}] = K[\lambda'_{p,\gamma}] \oplus \lambda'_{p,\gamma} \cdot \Delta_{\bar{p}}, \\ M'_j[\lambda'_{p,\gamma}] = K'_j[\lambda'_{p,\gamma}] \oplus \lambda'_{p,\gamma} \cdot \nabla_{\bar{p},j} \quad \forall j \in [\kappa]. \end{cases}$$

5. Eventually, we need the labels for encrypting and decrypting purposes. In particular, for $w \in \mathcal{I}_{\mathcal{A}} \cup \mathcal{I}_{\mathcal{B}} \cup \mathcal{I}_{\mathcal{W}}$, $\mathcal{S}_{\text{zk}}$ computes $\mathbb{L}_{w,0} \xleftarrow{\$} \mathbb{F}_{2^{\tau}}$ as specified in $\Pi_{\text{pa-2PC-pre}}$ (c.f. Figure 9).

The case of AND gates. As said above, we do not care whether $(\lambda_{A,\alpha} \oplus \lambda_{B,\alpha}) \cdot (\lambda_{A,\beta} \oplus \lambda_{B,\beta}) = \lambda'_{A,\gamma} \oplus \lambda'_{B,\gamma}$. Hence, in this case, the simulation makes the simulated proofs for checking AND gates indistinguishable from the real proofs. Recall that $L = W \cdot \text{bs} + \text{rm}$. Having permutations $(\pi_j)_{j \in [\kappa]}$, $\mathcal{S}_{zk}$ works as follows.

For $j \in [\kappa]$ and $i \in [L]$, we consider the following two sub-cases.

1. If $i \in [(k-1) \cdot W + 1, k \cdot W]$ for $k \in [\text{bs}]$, in the real protocol $\Pi_{\text{pa-2PC-eval}}$ (c.f. Figure 10), $\mathcal{P}_A$ and $\mathcal{P}_B$ use authenticated values $(a_{A,j}^{(i)}, b_{A,j}^{(i)}, c_{A,j}^{(i)})$ and $(a_{B,j}^{(i)}, b_{B,j}^{(i)}, c_{B,j}^{(i)})$, respectively, for testing whether $(\lambda_{A,\alpha} \oplus \lambda_{B,\alpha}) \cdot (\lambda_{A,\beta} \oplus \lambda_{B,\beta}) = \lambda'_{A,\gamma} \oplus \lambda'_{B,\gamma}$ for the $(i - (k-1) \cdot W)$ -th AND gate $(\alpha, \beta, \gamma, \wedge)$ by using protocol $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$ (c.f. Figure 5). However, as said above, $\mathcal{S}_{zk}$ does not generate $(\lambda_{A,\alpha}, \lambda_{A,\beta}, \lambda'_{A,\gamma})$ and $(\lambda_{B,\alpha}, \lambda_{B,\beta}, \lambda'_{B,\gamma})$ satisfying such condition. Therefore, to encompass the checking in sub-protocol $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$, $\mathcal{S}_{zk}$ first samples $a_{A,j}^{(i)}, b_{A,j}^{(i)}, a_{B,j}^{(i)}, b_{B,j}^{(i)} \xleftarrow{\$} \mathbb{F}_2$. Then, $\mathcal{S}_{zk}$ samples $(c_{A,j}^{(i)}, c_{B,j}^{(i)})$ satisfying

$$(c_{A,j}^{(i)} \oplus \lambda'_{A,\gamma} \oplus d_j^{(i)} \cdot b_{A,j}^{(i)} \oplus e_j^{(i)} \cdot a_{A,j}^{(i)}) \oplus (c_{B,j}^{(i)} \oplus \lambda'_{B,\gamma} \oplus d_j^{(i)} \cdot b_{B,j}^{(i)} \oplus e_j^{(i)} \cdot a_{B,j}^{(i)}) \oplus d_j^{(i)} \cdot e_j^{(i)} = 0$$

where $d_j^{(i)} = \lambda'_{A,\alpha} \oplus a_{A,j}^{(i)} \oplus \lambda'_{B,\alpha} \oplus a_{B,j}^{(i)}$ and $e_j^{(i)} = \lambda'_{A,\beta} \oplus b_{A,j}^{(i)} \oplus \lambda'_{B,\beta} \oplus b_{B,j}^{(i)}$.

2. If $i \in [\text{bs} \cdot W + 1, \text{bs} \cdot W + \text{rm}]$, $\mathcal{S}_{zk}$ simply samples $a_{A,j}^{(i)}, b_{A,j}^{(i)}, a_{B,j}^{(i)}, b_{B,j}^{(i)} \xleftarrow{\$} \mathbb{F}_2$. Then, $\mathcal{S}_{zk}$ samples $(c_{A,j}^{(i)}, c_{B,j}^{(i)})$ satisfying

$$(a_{A,j}^{(i)} \oplus a_{B,j}^{(i)}) \cdot (b_{A,j}^{(i)} \oplus b_{B,j}^{(i)}) = c_{A,j}^{(i)} \oplus c_{B,j}^{(i)}.$$

Having $(a_{A,j}^{(\ell)}, b_{A,j}^{(\ell)}, c_{A,j}^{(\ell)}, a_{B,j}^{(\ell)}, b_{B,j}^{(\ell)}, c_{B,j}^{(\ell)})_{j \in [\kappa], \ell \in [L]}$ sampled and computed according to the above process, $\mathcal{S}_{zk}$ can easily compute $(\tilde{a}_{A,j}^{(\ell)}, \tilde{b}_{A,j}^{(\ell)}, \tilde{c}_{A,j}^{(\ell)}, \tilde{a}_{B,j}^{(\ell)}, \tilde{b}_{B,j}^{(\ell)}, \tilde{c}_{B,j}^{(\ell)})_{j \in [\kappa], \ell \in [L]}$ such that, for $j \in [\kappa]$ and $\ell \in [L]$, it holds that

$$(a_{A,j}^{(\ell)}, b_{A,j}^{(\ell)}, c_{A,j}^{(\ell)}, a_{B,j}^{(\ell)}, b_{B,j}^{(\ell)}, c_{B,j}^{(\ell)}) = (\tilde{a}_{A,j}^{(\pi_j(\ell))}, \tilde{b}_{A,j}^{(\pi_j(\ell))}, \tilde{c}_{A,j}^{(\pi_j(\ell))}, \tilde{a}_{B,j}^{(\pi_j(\ell))}, \tilde{b}_{B,j}^{(\pi_j(\ell))}, \tilde{c}_{B,j}^{(\pi_j(\ell))})$$

by leveraging the bijections of permutations $(\pi_j)_{j \in [\kappa]}$.

Finally, $\mathcal{S}_{zk}$ computes the VOLEitH tags $M'_j[\tilde{a}_{p,j}^{(\ell)}], M'_j[\tilde{b}_{p,j}^{(\ell)}], M'_j[\tilde{c}_{p,j}^{(\ell)}]$ such that

$$\begin{cases} M'_j[\tilde{a}_{p,j}^{(\ell)}] = K'_j[\tilde{a}_{p,j}^{(\ell)}] \oplus \tilde{a}_{p,j}^{(\ell)} \cdot \nabla_{\bar{p},j}, \\ M'_j[\tilde{b}_{p,j}^{(\ell)}] = K'_j[\tilde{b}_{p,j}^{(\ell)}] \oplus \tilde{b}_{p,j}^{(\ell)} \cdot \nabla_{\bar{p},j}, \\ M'_j[\tilde{c}_{p,j}^{(\ell)}] = K'_j[\tilde{c}_{p,j}^{(\ell)}] \oplus \tilde{c}_{p,j}^{(\ell)} \cdot \nabla_{\bar{p},j}. \end{cases}$$

With the above strategy, we can see that the simulated proof is indistinguishable from the real one.

Knowledge soundness of $\Pi_{\text{pa-2PC}}$. To show that protocol $\Pi_{\text{pa-2PC}}$ is knowledge-sound, we construct an extractor $\mathcal{E}$ acting as a verifier $\mathcal{V}$ and interacting with $\mathcal{P}_A$ and $\mathcal{P}_B$ to extract their witnesses. $\mathcal{E}$ additionally can rewind both $\mathcal{P}_A$ and $\mathcal{P}_B$ to change the randomness as follows.

1. *Initial execution.* Initially, $\mathcal{E}$ acts as an honest $\mathcal{V}$ to run $\Pi_{\text{pa-2PC}}$ (by sequentially running $\Pi_{\text{pa-2PC-pre}}$, $\Pi_{\text{pa-2PC-eval}}$, and $\Pi_{\text{pa-2PC-ver}}$).
2. *Execution after rewinding.* Then, $\mathcal{E}$ rewinds $\Pi_{\text{pa-2PC}}$ to step 8 of $\Pi_{\text{pa-2PC-eval}}$ when both parties access $\mathcal{F}_{\text{svOLE-2PC}}$ (c.f. Figure 6). At this time, $\mathcal{E}$ samples new VOLEitH global keys and interacts with both $\mathcal{P}_A$ and $\mathcal{P}_B$ w.r.t. the newly sampled keys.

We assume that the VOLEitH global keys step 1 are denoted by $(\nabla_{A,j}^{(0)}, \nabla_{B,j}^{(0)})_{j \in [\kappa]}$. The ones for step 2 are $(\nabla_{A,j}^{(1)}, \nabla_{B,j}^{(1)})_{j \in [\kappa]}$. We now proceed with the extraction as follows.

Extracting randomness for masking. In step 8 (c.f. Figure 10), for $t \in \{0, 1\}$ and $p \in \{A, B\}$, we write

$$\begin{aligned} & (\nabla_{p,j}^{(t)})_{j \in [\kappa]}, (K_j^{(t)}[\lambda_{p,w}])_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (K_j^{(t)}[\lambda'_{p,w}])_{w \in \mathcal{I}_W}, \\ & (K_j^{(t)}[\tilde{a}_{p,j}^{(\ell)}], K_j^{(t)}[\tilde{b}_{p,j}^{(\ell)}], K_j^{(t)}[\tilde{c}_{p,j}^{(\ell)}])_{j \in [\kappa], \ell \in [L]} \end{aligned} \tag{9}$$

to imply that

- $t = 0$ is for those of the initial execution of $\mathcal{E}$, and
- $t = 1$ is for those of the execution after rewinding of $\mathcal{E}$.

For any $j \in [\kappa]$, since $\nabla_{\mathbf{B},j}^{(0)}$ and $\nabla_{\mathbf{B},j}^{(1)}$ are sampled independently from $\mathbb{F}_{2^\tau}$ in two different executions, with overwhelming probability $\frac{2^\tau-1}{2^\tau}$, $\nabla_{\mathbf{B},j}^{(0)} \neq \nabla_{\mathbf{B},j}^{(1)}$. Hence, we can instruct $\mathcal{E}$ to extract witnesses with the extraction idea as follows. Assume that $r \in \{0,1\}$ is authenticated such that

$$\begin{cases} M'_j[r] = K'_j{}^{(0)}[r] \oplus r \cdot \nabla_{\mathbf{B},j}^{(0)}, \\ M'_j[r] = K'_j{}^{(1)}[r] \oplus r \cdot \nabla_{\mathbf{B},j}^{(1)} \end{cases} \iff K'_j{}^{(0)}[r] \oplus r \cdot \nabla_{\mathbf{B},j}^{(0)} = K'_j{}^{(1)}[r] \oplus r \cdot \nabla_{\mathbf{B},j}^{(1)}.$$

If $\nabla_{\mathbf{A},j}^{(0)} \neq \nabla_{\mathbf{A},j}^{(1)}$, then we can compute r by simple algebra techniques. Hence, with this idea, $\mathcal{E}$ can extract the value mentioned above.

Hence, according to (9), we can extract

$$(\lambda_{p,w})_{w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{I}_W}, (\lambda'_{p,w})_{w \in \mathcal{I}_W}, (\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{p \in \{\mathbf{A}, \mathbf{B}\}, j \in [\kappa], \ell \in [L]}.$$

Extracting inputs. In step 5 (c.f. Figure 10), $\mathcal{P}_A$ and $\mathcal{P}_B$ published $\hat{z}_w$ for $w \in \mathcal{I}_A \cup \mathcal{I}_B$. Hence, $\mathcal{E}$ can compute inputs $z_w := \hat{z}_w \oplus \lambda_{A,w} \oplus \lambda_{B,w}$ for $w \in \mathcal{I}_A \cup \mathcal{I}_B$ where $\lambda_{A,w}$ and $\lambda_{B,w}$ are already extracted above.

Implications for XOR gates' satisfiability. As all parties compute $\hat{z}_\gamma := \hat{z}_\alpha \oplus \hat{z}_\beta$ for any XOR gate $(\alpha, \beta, \gamma, \oplus)$ (step 6 of Figure 10), we can imply the randomness for output of XOR gates as $\lambda_{A,\gamma} = \lambda_{A,\alpha} \oplus \lambda_{A,\beta}$ and $\lambda_{B,\gamma} = \lambda_{B,\alpha} \oplus \lambda_{B,\beta}$, and the actual value $z_\gamma = z_\alpha \oplus z_\beta$ once $\mathcal{E}$ knows $\lambda_{A,\alpha}, \lambda_{A,\beta}, \lambda_{B,\alpha}, \lambda_{B,\beta}, z_\alpha$, and z_β .

Satisfiability w.r.t. protocol $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$ (c.f. Figure 5). For simplicity, we first assume that, for $j \in [\kappa]$ and $\ell \in [L]$, it holds that

$$(a_{A,j}^{(\ell)} \oplus a_{B,j}^{(\ell)}) \cdot (b_{A,j}^{(\ell)} \oplus b_{B,j}^{(\ell)}) = c_{A,j}^{(\ell)} \oplus c_{B,j}^{(\ell)}.$$

With the computations and verifications in step 2 of $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$, we can imply that, for $p \in \{\mathbf{A}, \mathbf{B}\}, j \in [\kappa]$, and $i \in [W]$,

$$d_{p,j}^{(i)} = x_p^{(i)} \oplus a_{p,j}^{(i)}, \quad e_{p,j}^{(i)} = y_p^{(i)} \oplus b_{p,j}^{(i)}, \quad \tilde{z}_{p,j}^{(i)} = z_p^{(i)} \oplus c_{p,j}^{(i)} \oplus d_j^{(i)} \cdot b_{p,j}^{(i)} \oplus e_j^{(i)} \cdot a_{p,j}^{(i)}.$$

Notice that, in step 1 of $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$, every party computes $d_j^{(i)} := d_{A,j}^{(i)} \oplus d_{B,j}^{(i)}$ and $e_j^{(i)} := e_{A,j}^{(i)} \oplus e_{B,j}^{(i)}$, and checks whether $\tilde{z}_{A,j}^{(i)} \oplus \tilde{z}_{B,j}^{(i)} \oplus d_j^{(i)} \cdot e_j^{(i)} = 0$.

For simplicity, let $x^{(i)} := x_A^{(i)} \oplus x_B^{(i)}, y^{(i)} := y_A^{(i)} \oplus y_B^{(i)}, z^{(i)} := z_A^{(i)} \oplus z_B^{(i)}, a_j^{(i)} := a_{A,j}^{(i)} \oplus a_{B,j}^{(i)}, b_j^{(i)} := b_{A,j}^{(i)} \oplus b_{B,j}^{(i)}$, and $c_j^{(i)} := c_{A,j}^{(i)} \oplus c_{B,j}^{(i)}$. Hence, we see that

$$\begin{aligned} d_j^{(i)} &= d_{A,j}^{(i)} \oplus d_{B,j}^{(i)} = x_A^{(i)} \oplus a_{A,j}^{(i)} \oplus x_B^{(i)} \oplus a_{B,j}^{(i)} = x^{(i)} \oplus a_j^{(i)}, \\ e_j^{(i)} &= e_{A,j}^{(i)} \oplus e_{B,j}^{(i)} = y_A^{(i)} \oplus b_{A,j}^{(i)} \oplus y_B^{(i)} \oplus b_{B,j}^{(i)} = y^{(i)} \oplus b_j^{(i)}. \end{aligned}$$

Hence, it follows that

$$\begin{aligned} \tilde{z}_j^{(i)} &= \tilde{z}_{A,j}^{(i)} \oplus \tilde{z}_{B,j}^{(i)} \\ &= (z_A^{(i)} \oplus c_{A,j}^{(i)} \oplus d_j^{(i)} \cdot b_{A,j}^{(i)} \oplus e_j^{(i)} \cdot a_{A,j}^{(i)}) \oplus (z_B^{(i)} \oplus c_{B,j}^{(i)} \oplus d_j^{(i)} \cdot b_{B,j}^{(i)} \oplus e_j^{(i)} \cdot a_{B,j}^{(i)}) \\ &= z^{(i)} \oplus c_j^{(i)} \oplus d_j^{(i)} \cdot b_j^{(i)} \oplus e_j^{(i)} \cdot a_j^{(i)} \end{aligned}$$

Since $\tilde{z}_{A,j}^{(i)} \oplus \tilde{z}_{B,j}^{(i)} \oplus d_j^{(i)} \cdot e_j^{(i)} = 0$, it implies that

$$(z^{(i)} \oplus c_j^{(i)} \oplus d_j^{(i)} \cdot b_j^{(i)} \oplus e_j^{(i)} \cdot a_j^{(i)}) \oplus d_j^{(i)} \cdot e_j^{(i)} = 0.$$

By expressing $d_j^{(i)} = x^{(i)} \oplus a_j^{(i)}$ and $e_j^{(i)} = y^{(i)} \oplus b_j^{(i)}$, we see that the above equation is equivalent to

$$z^{(i)} \oplus c_j^{(i)} \oplus (x^{(i)} \oplus a_j^{(i)}) \cdot b_j^{(i)} \oplus (y^{(i)} \oplus b_j^{(i)}) \cdot a_j^{(i)} \oplus (x^{(i)} \oplus a_j^{(i)}) \cdot (y^{(i)} \oplus b_j^{(i)}) = 0.$$

It follows that

$$z^{(i)} \oplus c_j^{(i)} \oplus x^{(i)} \cdot y^{(i)} \oplus a_j^{(i)} \cdot b_j^{(i)} = 0.$$

As originally we assumed $(a_{A,j}^{(i)} \oplus a_{B,j}^{(i)}) \cdot (b_{A,j}^{(i)} \oplus b_{B,j}^{(i)}) = c_{A,j}^{(i)} \oplus c_{B,j}^{(i)}$. We see that $a_j^{(i)} \cdot b_j^{(i)} = c_j^{(i)}$. The above equation is equivalent to

$$z^{(i)} \oplus x^{(i)} \cdot y^{(i)} = 0.$$

Hence, we imply that

$$(x_A^{(i)} \oplus x_B^{(i)}) \cdot (y_A^{(i)} \oplus y_B^{(i)}) = z_A^{(i)} \oplus z_B^{(i)}.$$

Extracting randomness for arguing AND gates' satisfiability. From union bound, the probability that $\nabla_{B,j}^{(0)} \neq \nabla_{B,j}^{(1)} \forall j \in [\kappa]$ is at least $\frac{2^\tau - \kappa}{2^\tau}$ which is overwhelming by a careful setting of parameters τ and κ . In such a case, $\mathcal{E}$ can extract $(\tilde{a}_{p,j}^{(\ell)}, \tilde{b}_{p,j}^{(\ell)}, \tilde{c}_{p,j}^{(\ell)})_{j \in [\kappa], \ell \in [L]}$, for $p \in \{A, B\}$, by using the above way for each $j \in [\kappa]$.

We now consider $\Pi_{\text{pa-2PC-eval}}$ (c.f. Figure 10). Notice that, in steps 2 (of $\Pi_{\text{pa-2PC-eval}}$) and 1 (of $\Pi_{\text{pa-2PC-ver}}$), the random tuples are shuffled according to permutations $(\pi_j)_{j \in [\kappa]}$ provided by $\mathcal{V}$ in step 1. The actions of permuting those components following the “balls-and-bins” experiment [3,35] specified in Lemma 1 for testing the satisfaction $(\lambda_{A,\alpha} \oplus \lambda_{B,\alpha}) \cdot (\lambda_{A,\beta} \oplus \lambda_{B,\beta}) = \lambda'_{A,\gamma} \oplus \lambda'_{B,\gamma}$ for all AND gates $(\alpha, \beta, \gamma, \wedge)$. This is done by running steps 4 (of $\Pi_{\text{pa-2PC-eval}}$), and 2 and 4 (of $\Pi_{\text{pa-2PC-ver}}$). Hence, if both $\mathcal{P}_A$ and $\mathcal{P}_B$ cheat by having $(\lambda_{A,\alpha} \oplus \lambda_{B,\alpha}) \cdot (\lambda_{A,\beta} \oplus \lambda_{B,\beta}) \neq \lambda'_{A,\gamma} \oplus \lambda'_{B,\gamma}$ for some AND gate $(\alpha, \beta, \gamma, \wedge)$. This requires both $\mathcal{P}_A$ and $\mathcal{P}_B$ to have enough luck to win the experiment in Lemma 1 for all κ repetitions which has probability at most $\left(\frac{L}{\text{bs}}\right)^{-\kappa}$. Otherwise, cheating $\mathcal{P}_A$ and $\mathcal{P}_B$ can be detected with probability at least $1 - \left(\frac{L}{\text{bs}}\right)^{-\kappa}$ according to $\Pi_{\text{check-AND}}^{\mathcal{P}_A, \mathcal{P}_B, \mathcal{V}, \kappa, W}$, as analyzed above, and the testing in step 2 (c.f. $\Pi_{\text{pa-2PC-ver}}$).

Arguing AND gates' satisfiability. For each AND gate $(\alpha, \beta, \gamma, \wedge)$ as in step 6 (c.f. Figure 10), let $k = \hat{z}_\alpha + 2 \cdot \hat{z}_\beta$ be the index determined by $\mathcal{P}_B$. From here $\mathcal{P}_B$ publishes the decrypted tuple

$$(\lambda_{A,\gamma,k}, \mathbf{M}[\lambda_{A,\gamma,k}], (\mathbf{M}'[\lambda_{A,\gamma,k}])_{j \in [\kappa]}, \mathbf{L}_{\gamma,0} \oplus \mathbf{K}[\lambda_{B,\gamma,k}] \oplus \lambda_{A,\gamma,k} \cdot \Delta_A).$$

Notice that, from step 3 (of $\Pi_{\text{pa-2PC-ver}}$), $\mathcal{E}$ already determined

$$\begin{cases} \mathbf{K}_j^{(t)}[\lambda_{A,\gamma,k}] := \mathbf{K}_j^{(t)}[\lambda'_{A,\gamma}] \oplus \mathbf{K}_j^{(t)}[\lambda_{A,\gamma}] \oplus \hat{z}_\alpha \cdot \mathbf{K}_j^{(t)}[\lambda_{A,\beta}] \oplus \hat{z}_\beta \cdot \mathbf{K}_j^{(t)}[\lambda_{A,\alpha}], \\ \mathbf{K}_j^{(t)}[\lambda_{B,\gamma,k}] := \mathbf{K}_j^{(t)}[\lambda'_{B,\gamma}] \oplus \mathbf{K}_j^{(t)}[\lambda_{B,\gamma}] \oplus \hat{z}_\alpha \cdot \mathbf{K}_j^{(t)}[\lambda_{B,\beta}] \oplus \hat{z}_\beta \cdot \mathbf{K}_j^{(t)}[\lambda_{B,\alpha}] \oplus \hat{z}_\alpha \cdot \hat{z}_\beta \cdot \nabla_{B,j}. \end{cases}$$

for all $t \in \{0, 1\}$.

With a similar argument, we can extract $\lambda_{A,\gamma,k}$ and $\lambda_{B,\gamma,k}$. Moreover, they also satisfy

$$\begin{cases} \lambda_{A,\gamma,k} = \lambda'_{A,\gamma} \oplus \lambda_{A,\gamma} \oplus \hat{z}_\alpha \cdot \lambda_{A,\beta} \oplus \hat{z}_\beta \cdot \lambda_{A,\alpha}, \\ \lambda_{B,\gamma,k} = \lambda'_{B,\gamma} \oplus \lambda_{B,\gamma} \oplus \hat{z}_\alpha \cdot \lambda_{B,\beta} \oplus \hat{z}_\beta \cdot \lambda_{B,\alpha} \oplus \hat{z}_\alpha \cdot \hat{z}_\beta \end{cases} \quad (10)$$

according to the above system. According to step 6, $\mathcal{E}$ can determine $\hat{z}_\gamma := \lambda_{A,\gamma,k} \oplus \lambda_{B,\gamma,k}$. Let $\lambda'_\gamma := \lambda'_{A,\gamma} \oplus \lambda'_{B,\gamma}$, $\lambda_\gamma := \lambda_{A,\gamma} \oplus \lambda_{B,\gamma}$, $\lambda_\alpha := \lambda_{A,\alpha} \oplus \lambda_{B,\alpha}$, and $\lambda_\beta := \lambda_{A,\beta} \oplus \lambda_{B,\beta}$. Hence, we can transform (10) to be

$$\hat{z}_\gamma = \lambda'_\gamma \oplus \lambda_\gamma \oplus \hat{z}_\alpha \cdot \lambda_\beta \oplus \hat{z}_\beta \cdot \lambda_\alpha \oplus \hat{z}_\alpha \cdot \hat{z}_\beta.$$

By definition, $\hat{z}_\alpha = z_\alpha \oplus \lambda_\alpha$ and $\hat{z}_\beta = z_\beta \oplus \lambda_\beta$. Hence, the above equation is equivalent to

$$\hat{z}_\gamma = \lambda'_\gamma \oplus \lambda_\gamma \oplus (z_\alpha \oplus \lambda_\alpha) \cdot \lambda_\beta \oplus (z_\beta \oplus \lambda_\beta) \cdot \lambda_\alpha \oplus (z_\alpha \oplus \lambda_\alpha) \cdot (z_\beta \oplus \lambda_\beta).$$

It follows that

$$\hat{z}_\gamma = \lambda'_\gamma \oplus \lambda_\gamma \oplus z_\alpha \cdot z_\beta \oplus \lambda_\alpha \cdot \lambda_\beta.$$

As shown above, with probability at least $1 - \left(\frac{L}{\text{bs}}\right)^{-\kappa}$, $\lambda'_\gamma = \lambda_\alpha \cdot \lambda_\beta$. Hence, in such a case, $\hat{z}_\gamma = z_\alpha \cdot z_\beta \oplus \lambda_\gamma$. Since λ_γ was extracted above, we can easily guarantee that $\hat{z}_\gamma$ is a masked value of $z_\alpha \cdot z_\beta$ by λ_γ .

Step 7 (of $\Pi_{\text{pa-2PC-ver}}$) implies that the output $(z_w)_{w \in \mathcal{I}_W}$ is correct.

Soundness error. As we need a single rewinding to extract all potential witnesses as well as the constraints assuming $(a_{A,j}^{(\ell)} \oplus a_{B,j}^{(\ell)}) \cdot (b_{A,j}^{(\ell)} \oplus b_{B,j}^{(\ell)}) = c_{A,j}^{(\ell)} \oplus c_{B,j}^{(\ell)}$ holds for all $j \in [\kappa]$ and $\ell \in [L]$. We see that the soundness error for cheating $\mathcal{P}_A$ and $\mathcal{P}_B$ to break the scheme is $2^{-\kappa \cdot \tau}$ for all κ repetitions.

The use of balls-and-bins method (c.f. Lemma 1) incurs a soundness error $\left(\frac{L}{\text{bs}}\right)^{-\kappa}$.

Hence, the total soundness error is

$$2^{-\kappa \cdot \tau} + \left(\frac{L}{\text{bs}}\right)^{-\kappa} + \text{negl}(\kappa, \tau)$$

where $\text{negl}(\kappa, \tau)$ denotes the negligible probability for breaking the security of components in the protocol, e.g., commitments. $\square$